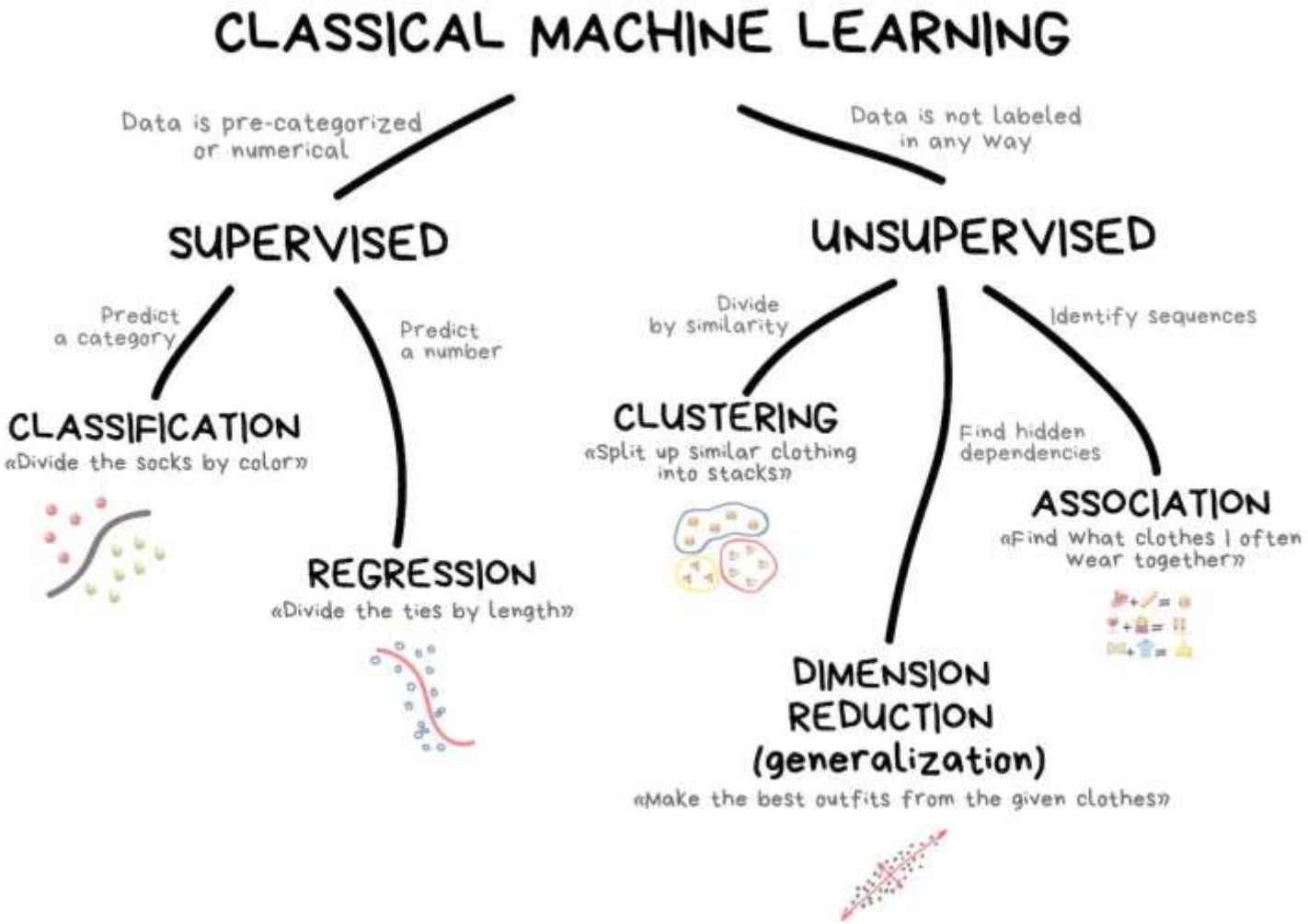
Data Science

Introduction to Machine Learning

Dr. Juan A. Ferrer-Bonsoms jafhernandez@unav.es

Tecnun Universidad de Navarra | ESCUELA DE INGENIERÍA INGENIARITZA ESKOLA SCHOOL OF ENGINEERING

Introduction to Machine Learning



Supervisado

El aprendizaje supervisado consiste en **predecir**.

Habitualmente, la entrada de datos es una "sábana de datos" tipo Excel.
En algunos casos, es necesario tener en cuenta relaciones entre datos (pixeles en una imagen), o trabajar con ellos hasta conseguir la "sábana" (text mining).

Regresión

- Una o varias variables continuas

Clasificación

- Etiquetas de diferentes clases
- Pueden predecirse varias clases a la vez ("multilabel classification")

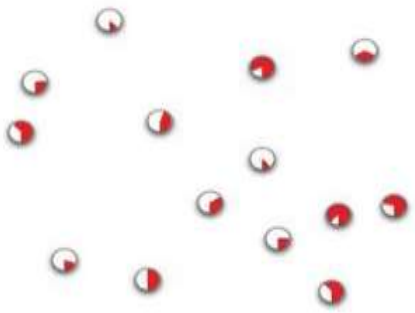
Otras

- Ordinal: predigo entre las clases que hay pero tienen un orden interno.

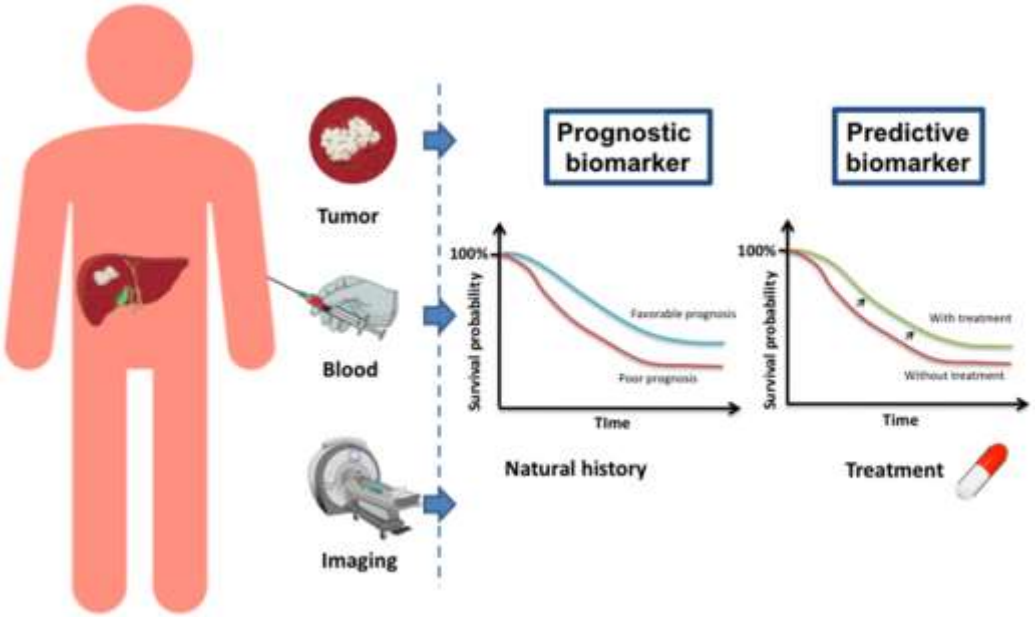
Supervisado

REGRESIÓN

La variable de interés a predecir es **cuantitativa**



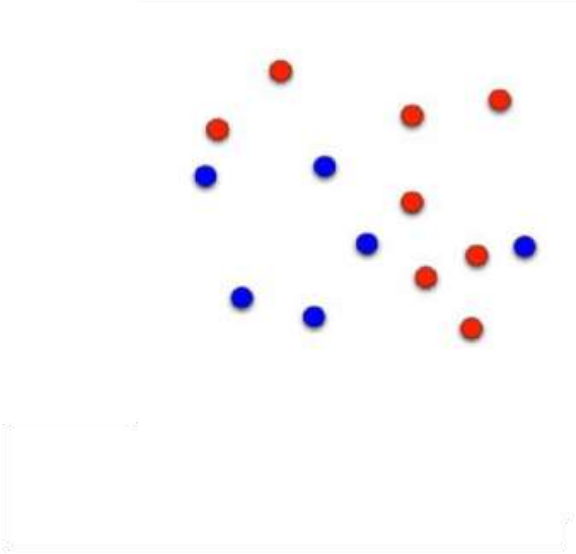
X_1 , X_2 , … , X_n	C
a , b , … , b	1.1
b , b , … , a	0.1
a , a , … , b	6.3
b , a , … , b	2.0
a , b , … , a	3.3
b , a , … , a	2.1
a , a , … , b	5.5
a , b , … , a	4.0
a , b , … , b	0.5
b , a , … , b	5.6
b , b , … , a	4.3
a , a , … , b	2.2
b , a , … , a	1.2
a , a , … , a	3.1



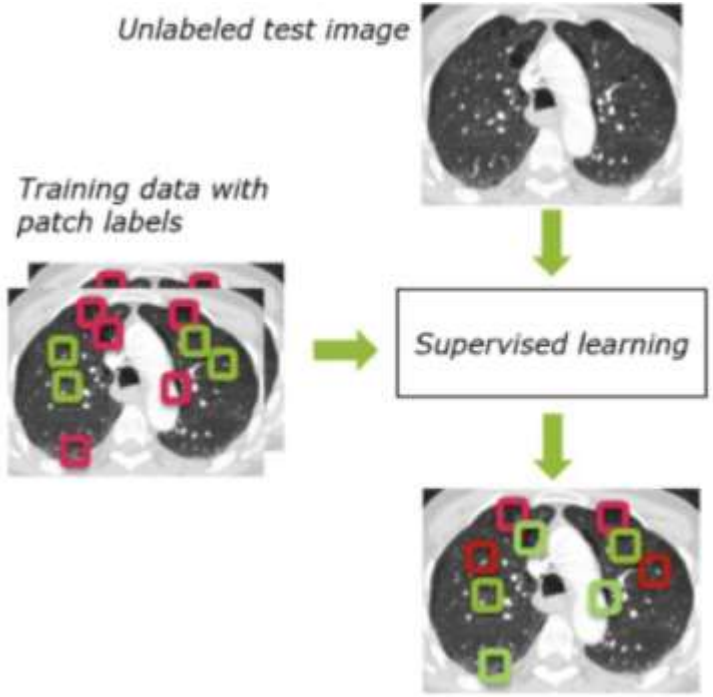
Clasificación supervisada (dos o más clases)

El algoritmo se entrena con una entrada que ha sido etiquetada para clases (**variables discreta**) concreta mutuamente excluyentes..

Si son varias clases, se denomina multiclase.



X_1 , X_2 , ... , X_n	C
a , b , ... , b	+
b , b , ... , a	-
a , a , ... , b	-
b , a , ... , b	+
a , b , ... , a	-
b , a , ... , a	-
a , a , ... , b	+
a , b , ... , a	-
a , b , ... , b	-
b , a , ... , b	-
b , b , ... , a	+
a , a , ... , b	+
b , a , ... , a	-
a , a , ... , a	+



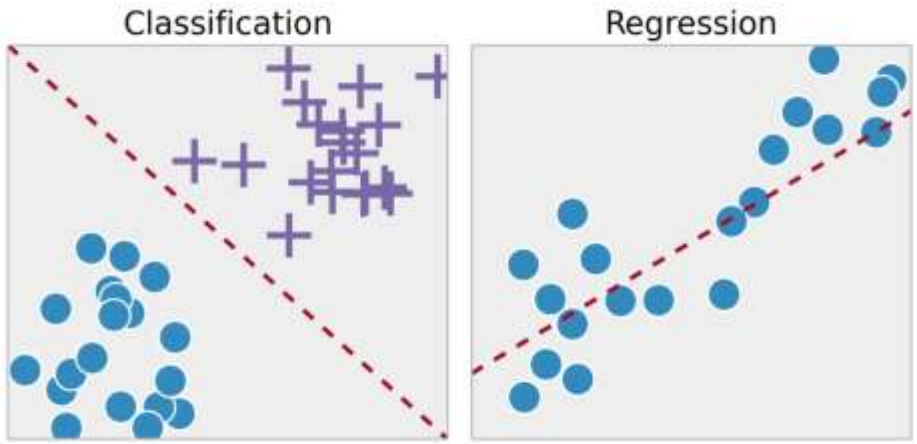
Supervisado

Y: output (dependent) variable (response, target)

X: input (independent) variables (features)

Predictive model: $Y = f(x) + \varepsilon$

Objective: find a useful approximation $\hat{f}(x)$ to the function $f(x)$ that captures the relationship between the inputs and output.



Supervisado

$$Regression: \begin{cases} MSE\ (Mean\ Square\ Error) & : \sum_{i=1}^{n} \frac{\left(y_i - \hat{f}(x_i)\right)^2}{n} \\ MAE\ (Mean\ Absolute\ Error) & : \sum_{i=1}^{n} \frac{\left|y_i - \hat{f}(x_i)\right|}{n} \\ Linear\ Correlation & \sum_{i=1}^{n} \frac{cov(y_i, \hat{f}(x_i))}{\sigma_y\ \sigma_{\hat{f}(x)}} \end{cases}$$

Others:

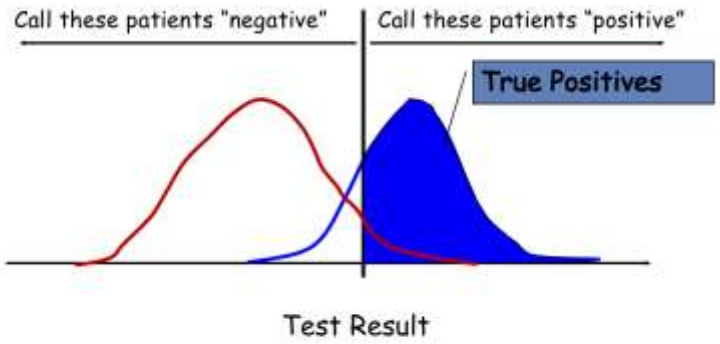
- RMSLE (Root mean squared logarithmic error)

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(log(y_i+1) - log(\hat{y}_i+1))^2)}$$

Supervisado

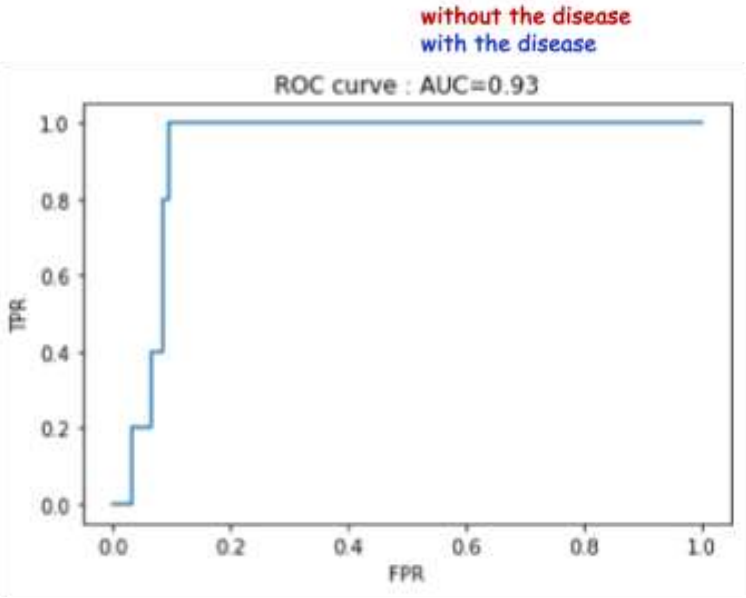
$$\boldsymbol{Classification} = \begin{cases} \boldsymbol{Accuracy}\ (\boldsymbol{0-1\ loss}): \displaystyle\sum_{i=1}^{n} \frac{I(y_i = \hat{f}(x_i))}{n} \\ where\ I\left(y_i = \hat{f}(x_i)\right) = \begin{cases} 1\ if\ \ y_i = \hat{f}(x_i) \\ 0\ if\ \ y_i \neq \hat{f}(x_i) \end{cases} \\ \\ \boldsymbol{Log-likelihood\ deviance}: \\ -2\displaystyle\sum_{i=1}^{n} \frac{\sum_{k=1}^{K} I(y_i = k) \cdot \log(\widehat{P_k}(x_i))}{n} \\ where\ I\left(y_i = \hat{f}(x_i)\right) = \begin{cases} 1\ if\ \ y_i = \hat{f}(x_i) \\ 0\ if\ \ y_i \neq \hat{f}(x_i) \end{cases} \end{cases}$$

Supervisado

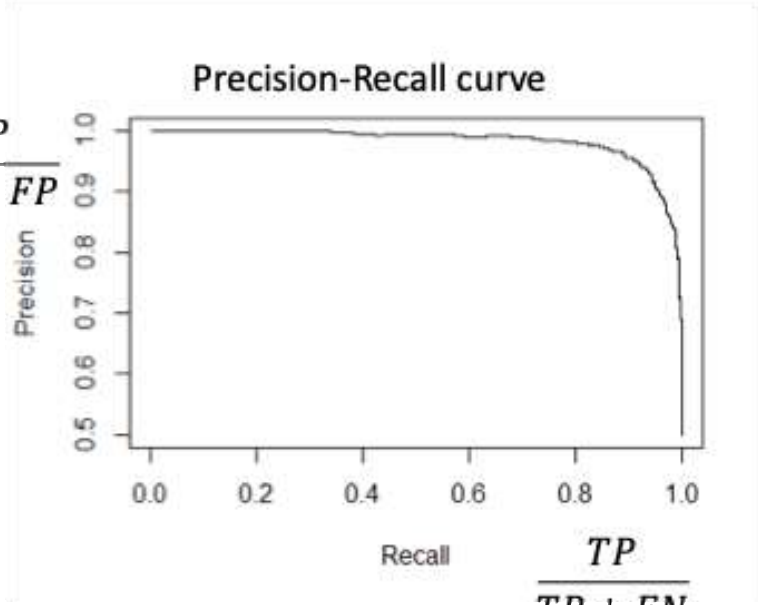


	predicted events	predicted non-events
actual events	True Positive	False Negative
actual non-events	False Positive	True Negative

$$TPR = \frac{TP}{TP + FN}$$



$$\frac{TP}{TP + FP}$$



$$\frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

Chicco and Jurman *BioData Mining* (2023) 16:4
https://doi.org/10.1186/s13040-023-00322-4

BioData Mining

$$\mathrm{MCC} = \frac{\mathrm{TP} \times \mathrm{TN} - \mathrm{FP} \times \mathrm{FN}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})(\mathrm{TN}+\mathrm{FP})(\mathrm{TN}+\mathrm{FN})}}$$

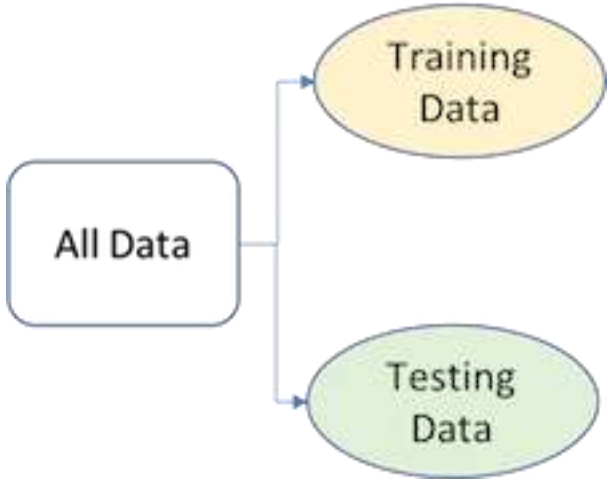
METHODOLOGY Open Access

The Matthews correlation coefficient (MCC) should replace the ROC AUC as the standard metric for assessing binary classification

Davide Chicco and Giuseppe Jurman

Model Performance

- The goodness of the predictive model cannot be assessed with the data used to fit the value of the parameters (**training data**).
- **Good performance of a model**: capacity of obtaining good results on **independent (test) data**.
- Training error is an optimistic estimate of **Expected test error**

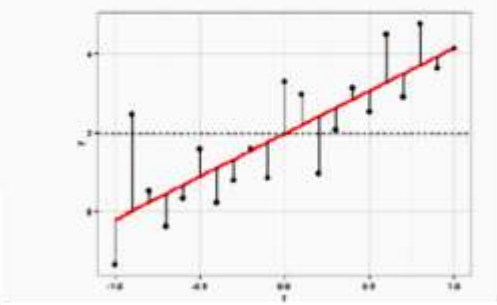


$\sum(y_i - f_i)^2 = 18.568$

$\sum(y_i - \bar{y})^2 = 55.001$

$R^2 = 1 - \frac{18.568}{55.001}$

$R^2 = 0.6624$

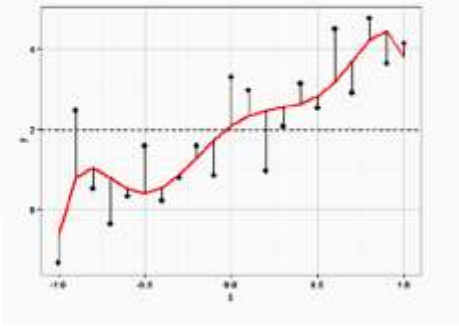


$\sum(y_i - f_i)^2 = 15.276$

$\sum(y_i - \bar{y})^2 = 55.001$

$R^2 = 1 - \frac{15.276}{55.001}$

$R^2 = 0.72$



Model Performance

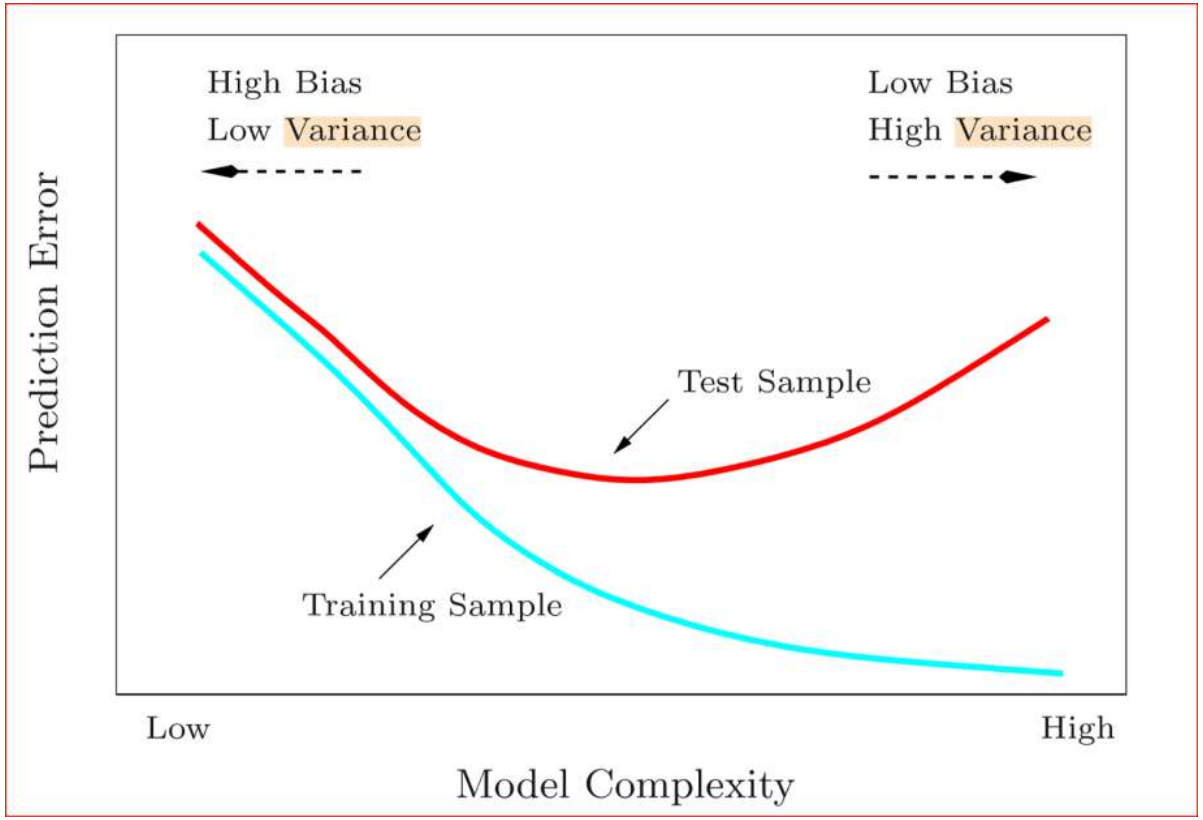
Bias-variance trade-off

$$Err(x_0) = E\left(Y - \hat{f}(x_0)\right)^2 = \sigma^2{}_\varepsilon + \text{Bias}^2 + \text{Variance}$$

Bias depends on training error

Variance on model complexity

Expected Test Error = Training error + Optimism

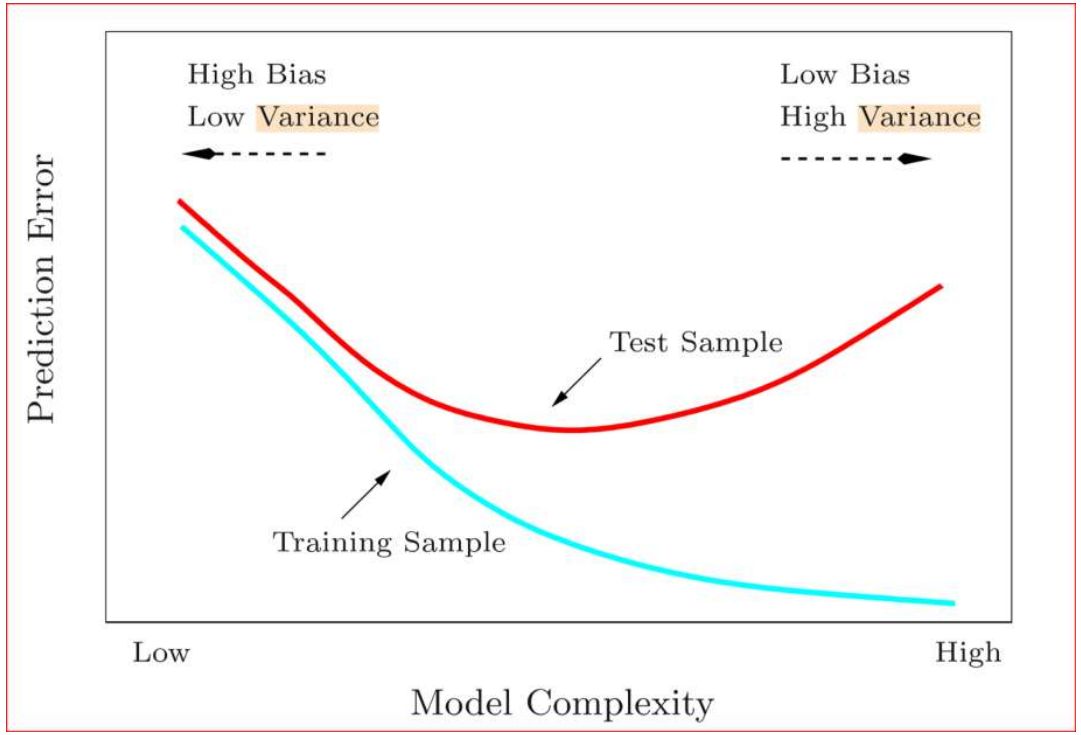


Model Performance

Bias-variance trade-off

$$Err(x_0) = E\left(Y - \hat{f}(x_0)\right)^2 = \sigma^2{}_\varepsilon + \text{Bias}^2 + \text{Variance}$$

- **Higher complexity** in the predictive model **reduces the training error**, since we have more degrees of freedom to explain the response (**overfitting**).
- This is not always translated to the test error. In many cases, rather the opposite.



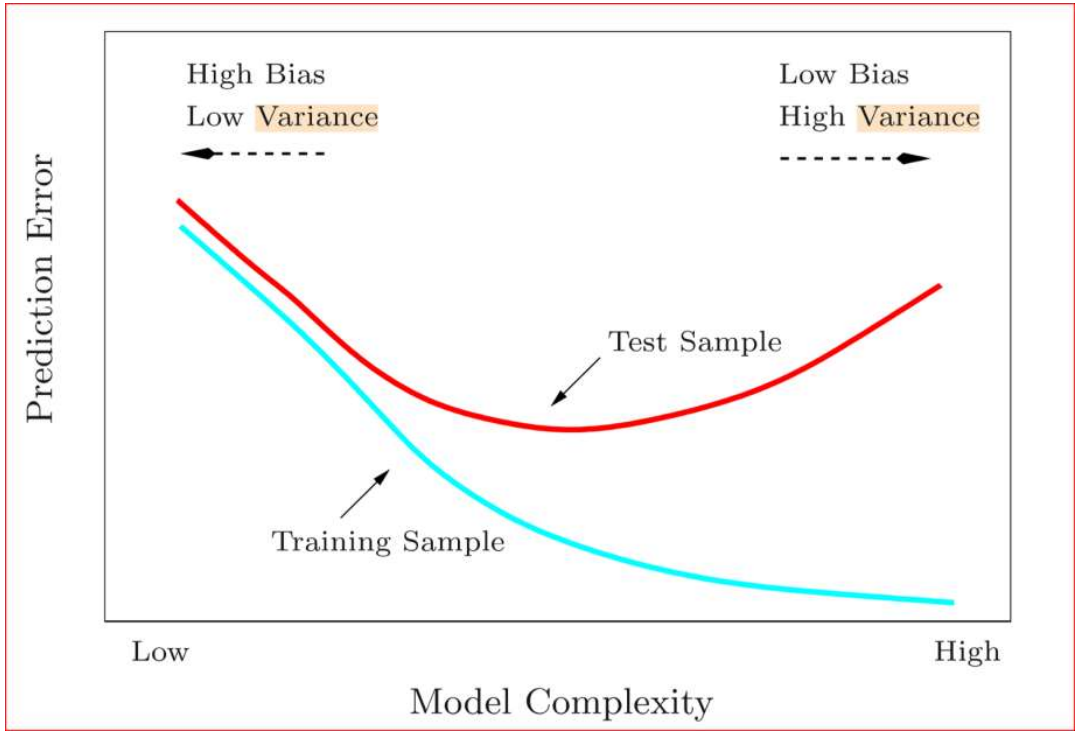
Model Performance

- To ponderate complexity (optimism in training error), we have different theoretical measures: **AIC (Akaike Information Criterion)** and **BIC (Bayesian Information Criterion)**

- **In linear regression:**

$$AIC = n \cdot \log(MSE) + 2 \cdot p$$

$$BIC = n \cdot \log(MSE) + \log(n) \cdot p$$

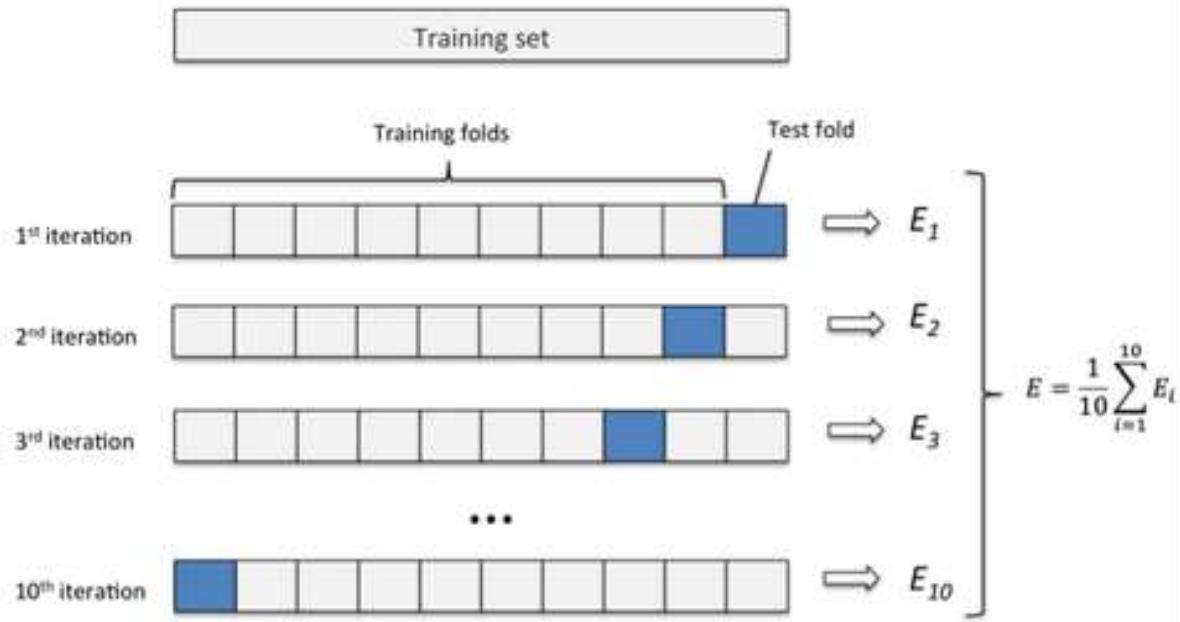


Model Performance

- **K-fold cross-validation:**

  - Data is split into *K* roughly equal parts
  - For the *k*-th part, we fit the model using (*k-1*) parts of the model and calculate the prediction error for the *k*-th part.
  - This is repeated for *k*=1,…,*K* and combines *K* estimates of prediction error, as follows:

$$CV\,(\hat{f}) = \frac{1}{K}\sum_{k=1}^{K}\frac{1}{N}\sum_{i=1}^{N}\mathrm{L}\left(y_i, \hat{f}^k(x_i)\right)$$



Model Performance (data_mvn.txt)

We have a dataset with 500 samples (**n**), 100 explanatory variables (**p**) and the response variable (**y**).

Assume that data from explanatory variables come from a multivariable normal distribution **N(μ, σ)**, where **μ_i=0 (i=1,…,p)** and **σ_{ij}= $\delta^{(i-j)}$ (i=1,…,p; j=1,…,p)**, where δ=0.5. **n** samples from this multivariate random variable was randomly generated and stored in the matrix **X** of dimensions **n x p**.

The response variable **y** comes from a multivariable normal distribution **N(μ, σ)**, where **μ=X*β** and **σ= s*I_{n*n}**. Now **β** stores the actual weights of explanatory variables, here **βi=1 for i≤5 and βi=0 for i>5.** On the other hand, **σ** is a diagonal matrix where **s= (β'*X* β)/v,** being **v** the signal-to-noise-ratio, here **v=0.5.**

Model Performance

Hyperparameters (aka *tuning parameters*) are the "knobs to twiddle"[7] to control the complexity of machine learning algorithms and, therefore, the bias-variance trade-off. Not all algorithms have hyperparameters (e.g., ordinary least squares[8]); however, most have at least one or more.

Ridge

$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}\beta_j^2$$

It has the effect of shrinking the estimates of βj towards zero

Lasso

$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}|\beta_j|$$

It has the effect of shrinking the estimates of βj **exactly** zero: variable selection.

Elastic-net

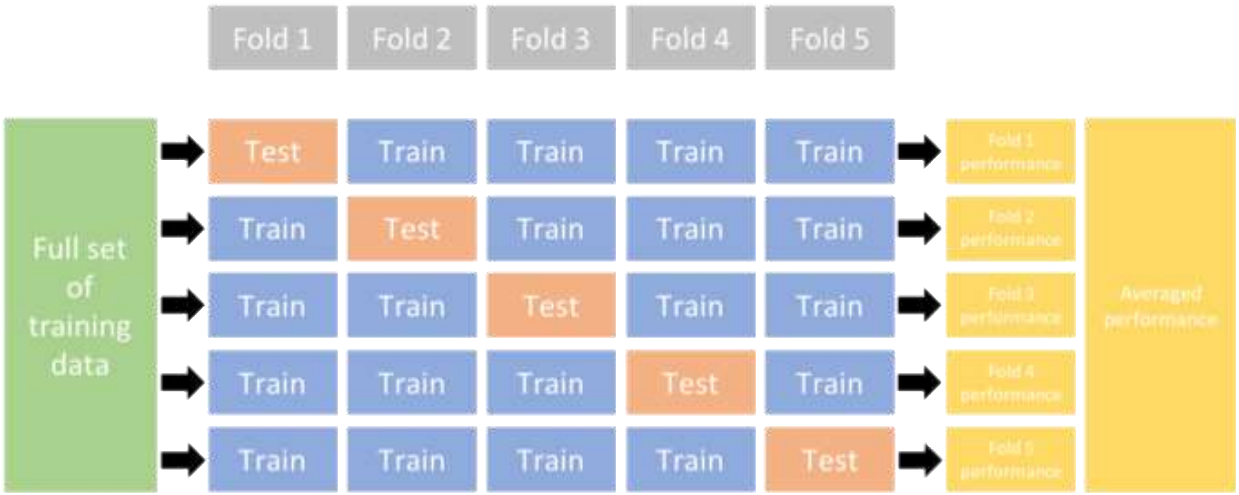
$$\frac{\sum_{i=1}^{n}(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij})^2}{2n}+\lambda(\alpha\sum_{j=1}^{p}|\beta_j|+\frac{1-\alpha}{2}+\sum_{j=1}^{p}\beta_j^2$$

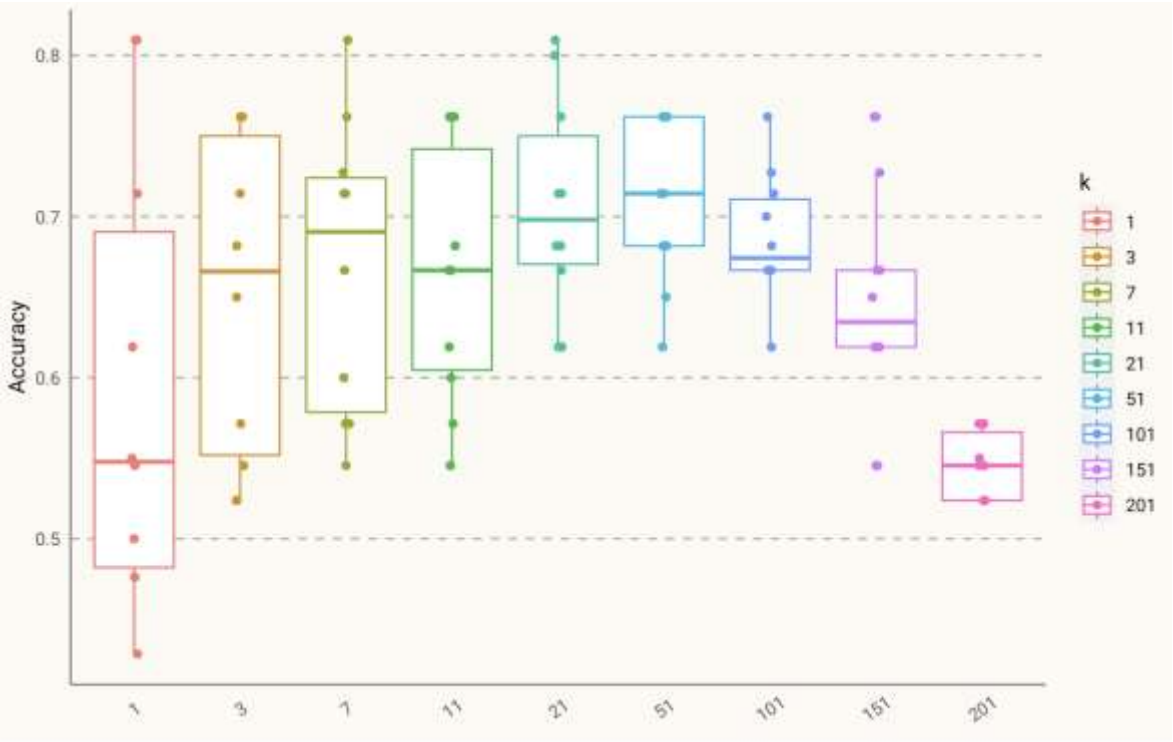
It is a combination of both Ridge and Lasso. If alpha = 0 we'll perform Ridge, and if alpha = 1 we'll perform lasso.

Model Performance

Predecir si una persona va a tener un ataque al corazón.
Seleccionamos un modelo, en este caso KNN.
Hyperparameter: la “k”: número de vecinos.
Seleccionamos varias opciones para la k: 1,3,7,11, …. , 201.

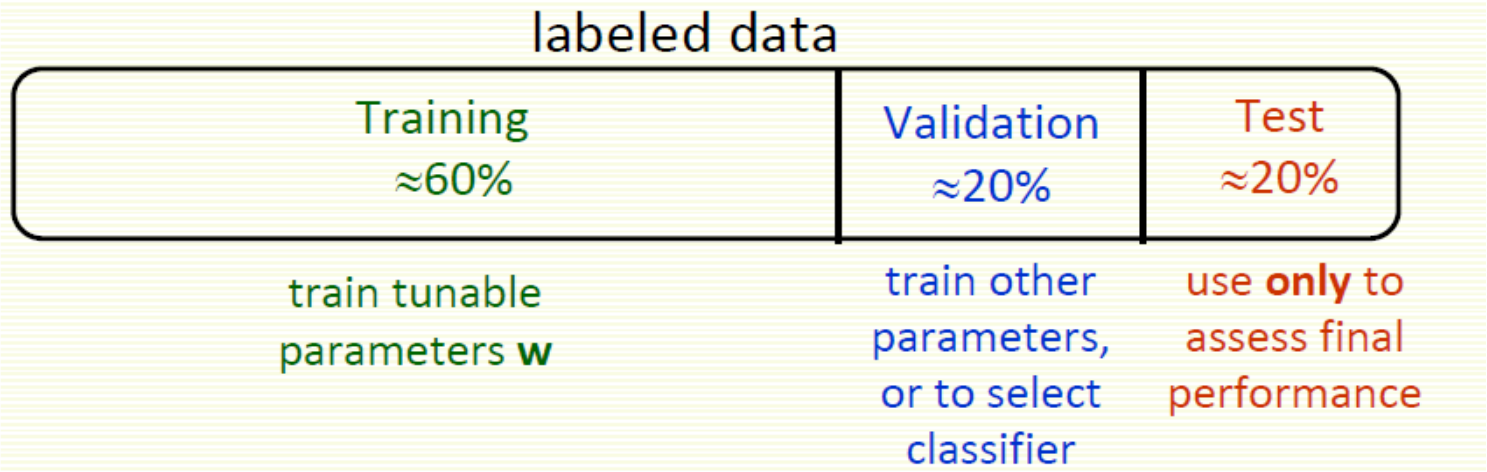
K-fold cross validation

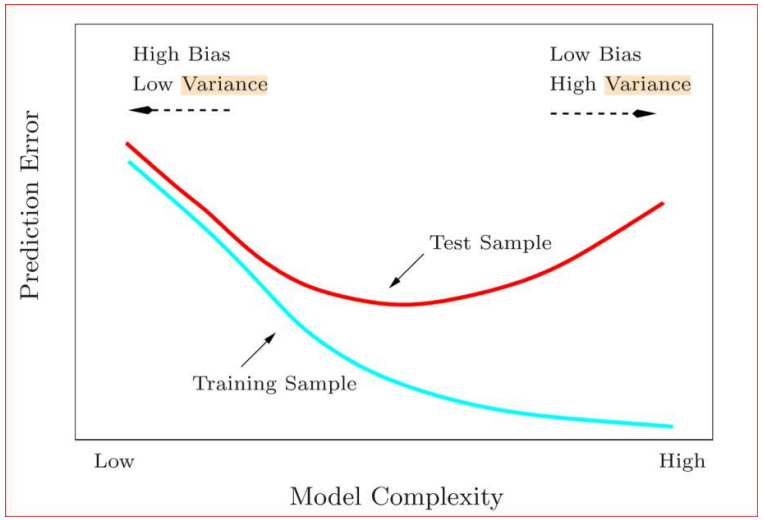




Aplico cross-validation a cada ”k” seleccionada

Model Performance





- Train data:
  - Used to train the parameters of the model.
- Validation data:
  - In many cases, used to find the "hyperparameters" of the model, e.g. to select the variables to include in the model.
  - How simple do I have to make my model?
- Test data:
  - **Locked data not used at all, that provide an accurate and unbiased quality for the model.**

Feature Engineering

La cantidad de datos disponible (en algunos casos) es abrumadora:

- En mi campo (los datos clínicos) especialmente: puede analizarse la expresión de 20.000 genes, o de 200.000 isoformas de esos genes, o de 2.000.000 de mutaciones en el genoma…
- En el análisis de imagen, cada pixel es un dato.
- En imagen médica (una tomografía, por ejemplo), cada voxel es una variable.

Es necesario seleccionar las variables adecuadas para cada problemas:

- ¿Cuáles son los variables de interés?

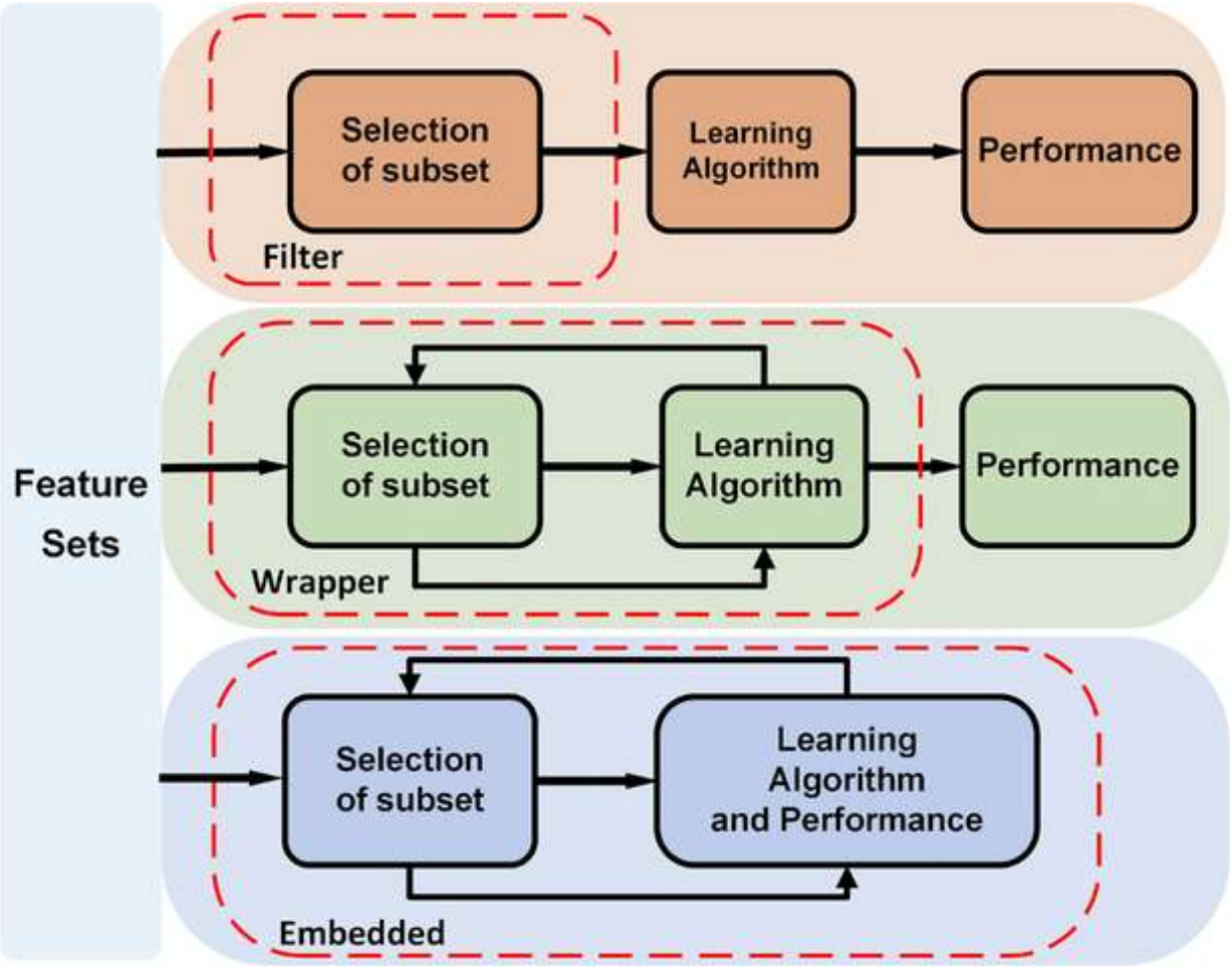
Si seleccionamos bien las variables:

- Funcionará más rápido
- Funcionará mejor
- Será más fácil de interpretar
- Generalizará mejor para nuevos casos

¡Es un paso clave!: si las variables están bien elegidas, casi cualquier algoritmo funcionará bien, si están mal elegidas, casi cualquiera funcionará mal.

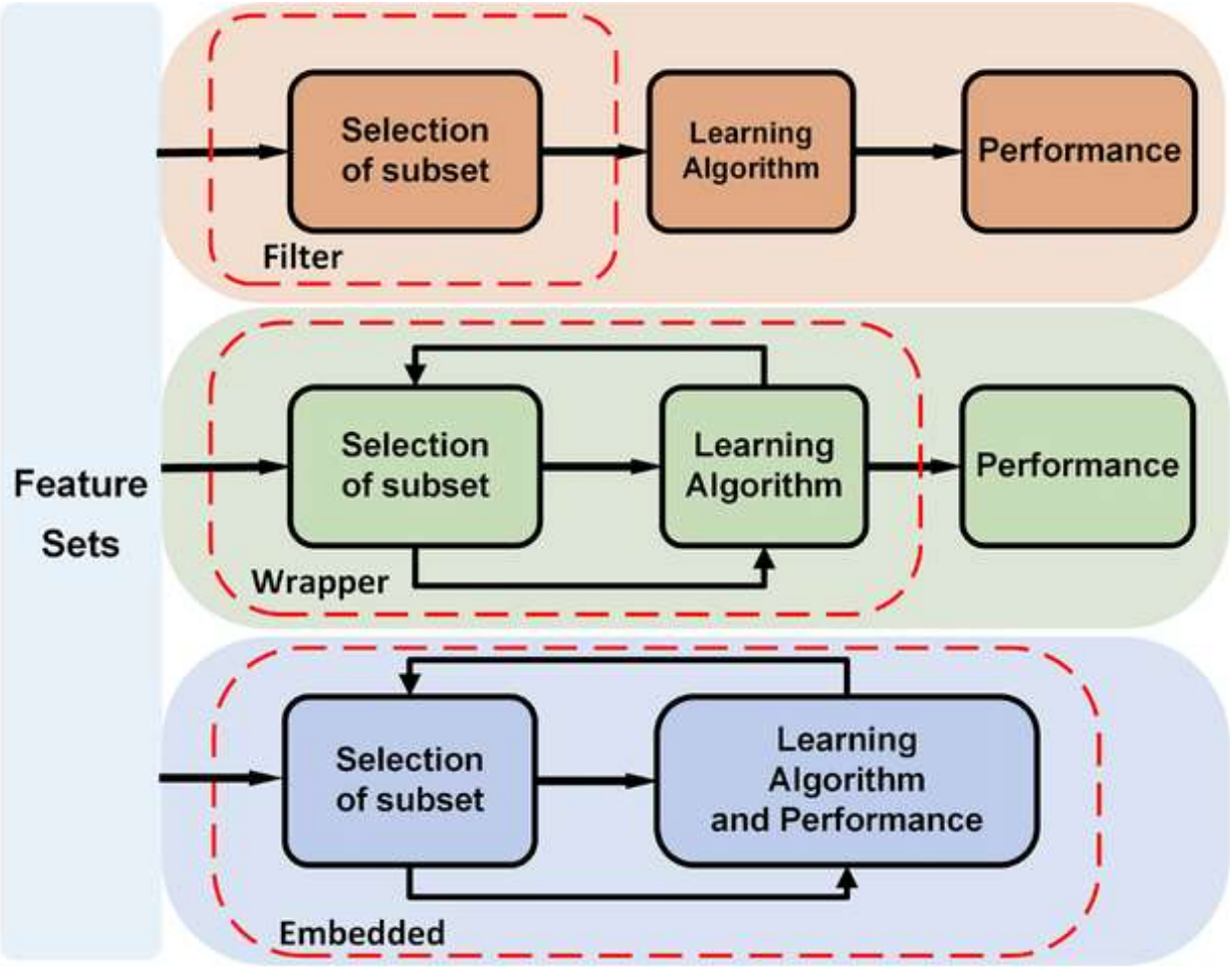
Feature Engineering

- **Filtros:**
  - Buscamos variables relacionadas con la respuesta que buscamos y filtramos las que suponemos que no influyen
    - Ejemplo: si un pixel en una imagen no cambia nunca podemos no considerarlo.
- **Wrappers:**
  - Seleccionan grupos de variables que se adaptan al problema que quiero resolver.
  - Se consigue de forma iterativa y, ordinariamente, **requieren mucho tiempo**.
    - Ejemplo: “Forward Selection”. De todas las variables, elijo aquella que mejor predice, con esa, busco otra con la que prediga mejor, con esas dos, busco una tercera…



Feature Engineering

- **Embebidos:**
  - Algunos algoritmos **seleccionan de forma automática** las variables más interesantes, es decir, no necesitan que se las seleccionemos nosotros.
  - Es el caso de la regresión LASSO, los arboles de clasificación y algunos tipos de Deep Learning.
  - Aunque no sea estrictamente necesario, **suele aplicarse un filtro antes** para acelerar el entrenamiento.



Ridge $$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}\beta_j^2$$

It has the effect of shrinking the estimates of βj towards zero

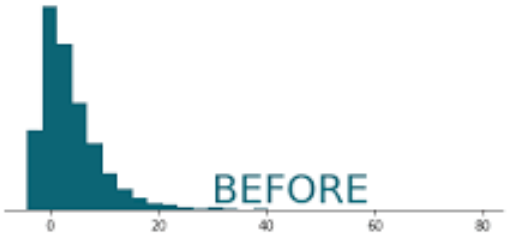
Feature Engineering

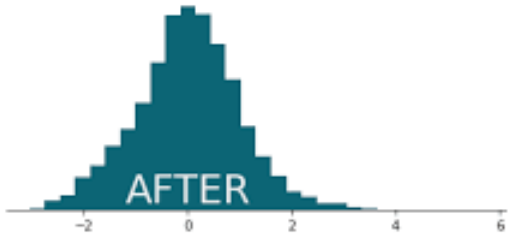
Also refers to the addition, deletion or transformation of data.

It requires you to spend time understanding your data!

- Target engineering: transforming the response variable if needed (not always)
  - Log transformation
  - Box cox transformation

- Dealing with missingness
  - Imputation:
    - Estimated statistic (mean, median, etc.)
    - K-nearest neighbor
    - Tree-based





- Feature filtering
  - Zero and near-zero variance variables.

- Numeric feature engineering: Like the process of Target engineering.

Machine Learning supervised review

Steps BEFORE modelling: **EXPLORATORY DATA ANALYSIS**

1. Define the problem: what do we want to predict? Which data is available?
2. Explore and understand the data: visualization and hypothesis.

Steps FOR modeling:

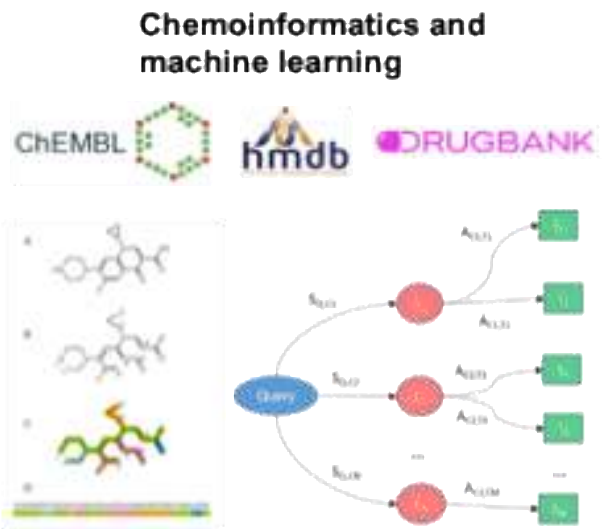
1. **Division of data in training and testing:** strategy to evaluate the model.

2. **Feature and Target engineering:** apply the necessary transformations.

3. **Select the model**
   - Cross-validation
   - Hyperparameter optimization
   - Compare models

4. **Evaluate the models**: make the prediction and error in the Test set.

Ejercicio 2

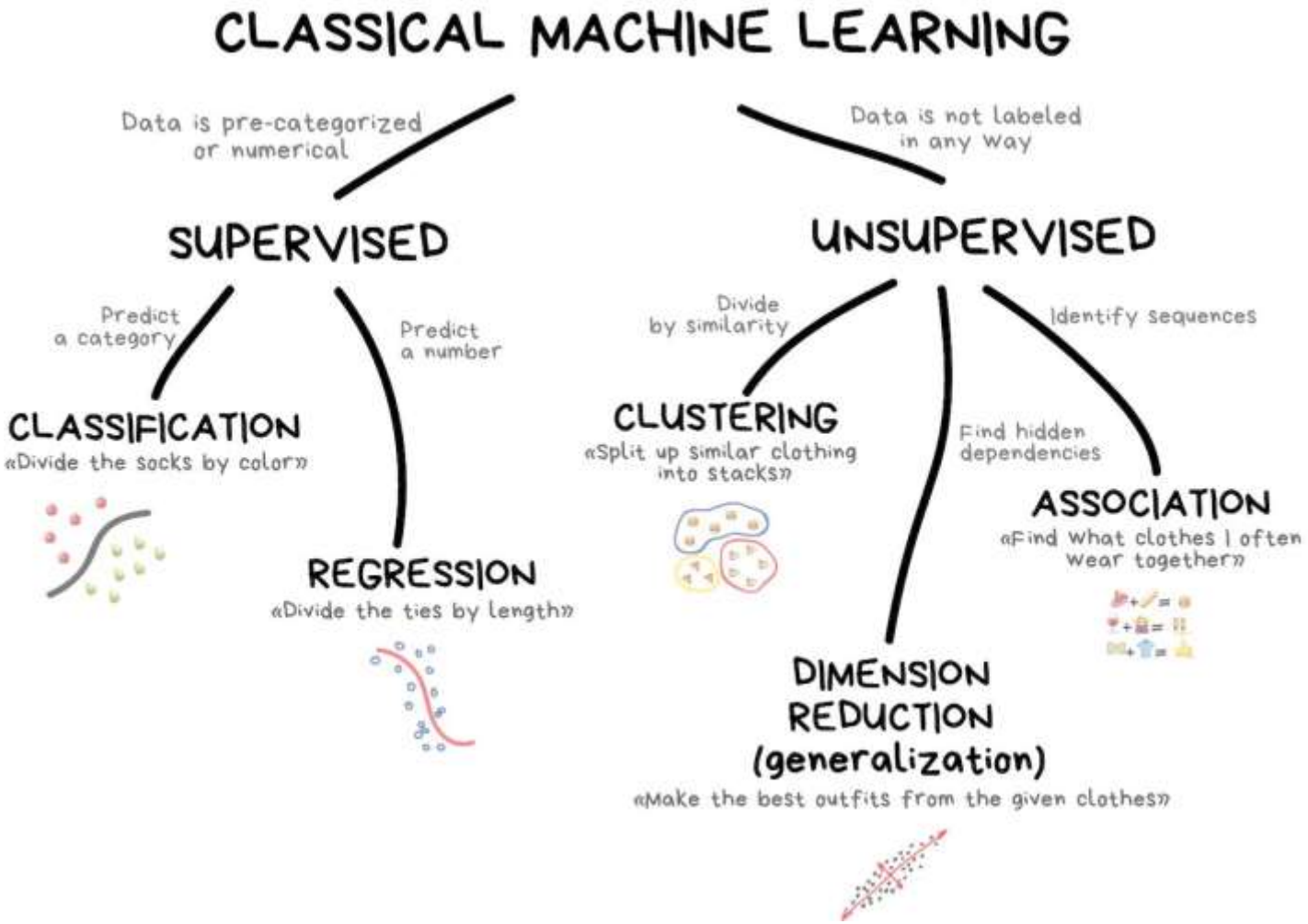


Information for a database of molecules as to their teratogenic potential. Chemical and structural descriptors of molecules.

Objectives:

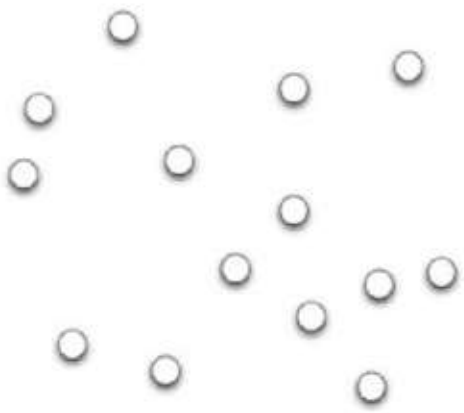
1. Construct a Logistic Regression model with L1 regularization (Lasso).
2. Construct a Logistic Regression with elasticnet regularization.
3. Construct a Logistic Regression using SelectKbest feature selection strategy

Introduction to Machine Learning



CLUSTERING

(Algoritmos de Agrupamiento)



X_1 , X_2 , ... , X_n	C
a , b , ... , b	?
b , b , ... , a	?
a , a , ... , b	?
b , a , ... , b	?
a , b , ... , a	?
b , a , ... , a	?
a , a , ... , b	?
a , b , ... , a	?
a , b , ... , b	?
b , a , ... , b	?
b , b , ... , a	?
a , a , ... , b	?
b , a , ... , a	?
a , a , ... , a	?

Dada una colección de registros/muestras (conjunto de entrenamiento)...

- Cada registro contiene un conjunto de atributos/características/predictores
- No hay una "característica objetivo" (clase) que supervise el proceso de aprendizaje

Encontrar grupos con:

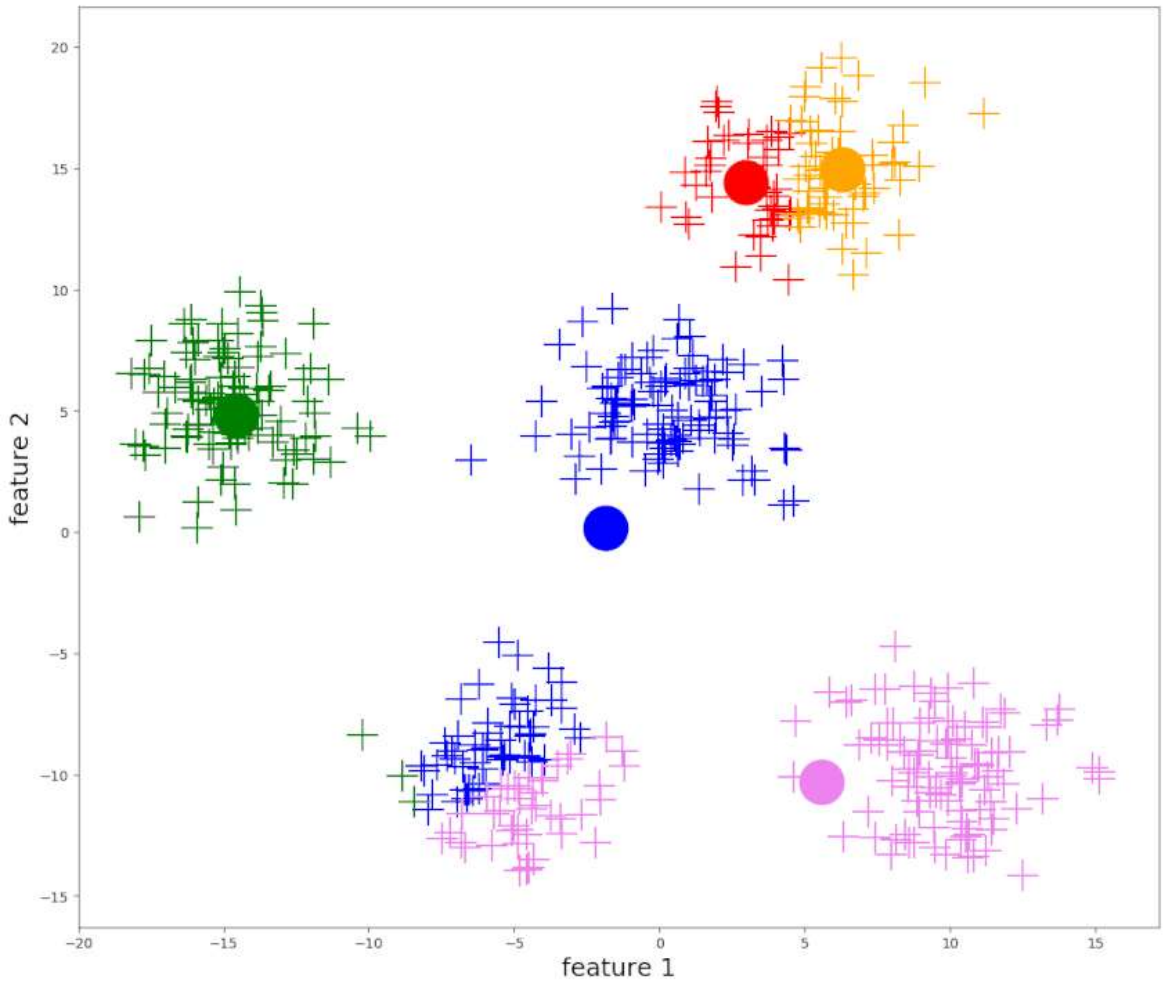
- Gran *homogeneidad intragrupo*: agrupación de muestras similares
- Gran *heterogeneidad intergrupos*

CLUSTERING

- Encontrar dentro de los datos de entrada, grupos de empleados/máquinas/usuarios/productos que tengan un patrón similar entre ellos y diferente al resto.
  - ¿Qué es un patrón? → **Es clave elegir bien la vara de medir.**
  - ¿Cuántos grupos?
  - ¿Deben pertenecer todos los elementos a un grupo o puede haber outliers?
  - Los grupos, ¿deben tener la misma forma o pueden ser de diferente forma?

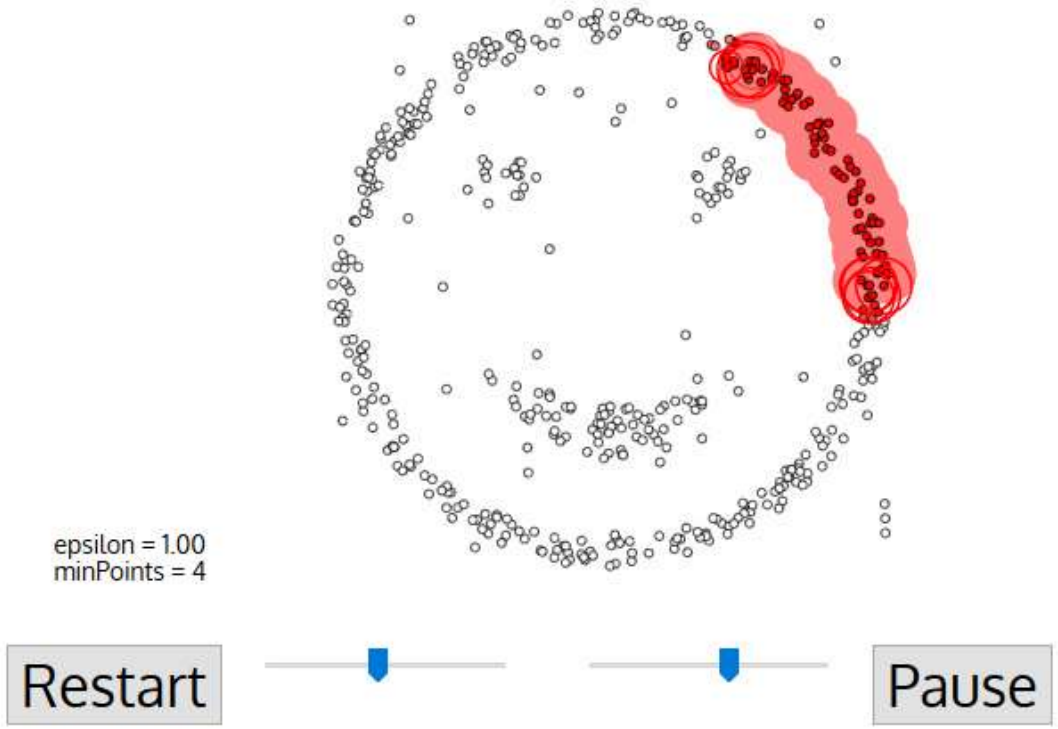
CLUSTERING

- Algunos algoritmos se basan en comparar las coordenadas de cada punto y asignan cada punto al cluster más cercano según esa distancia
  - K-means, K-medians, K-medoids.
- Hay algunas variantes que pueden seleccionar de forma automática el número de clusters.

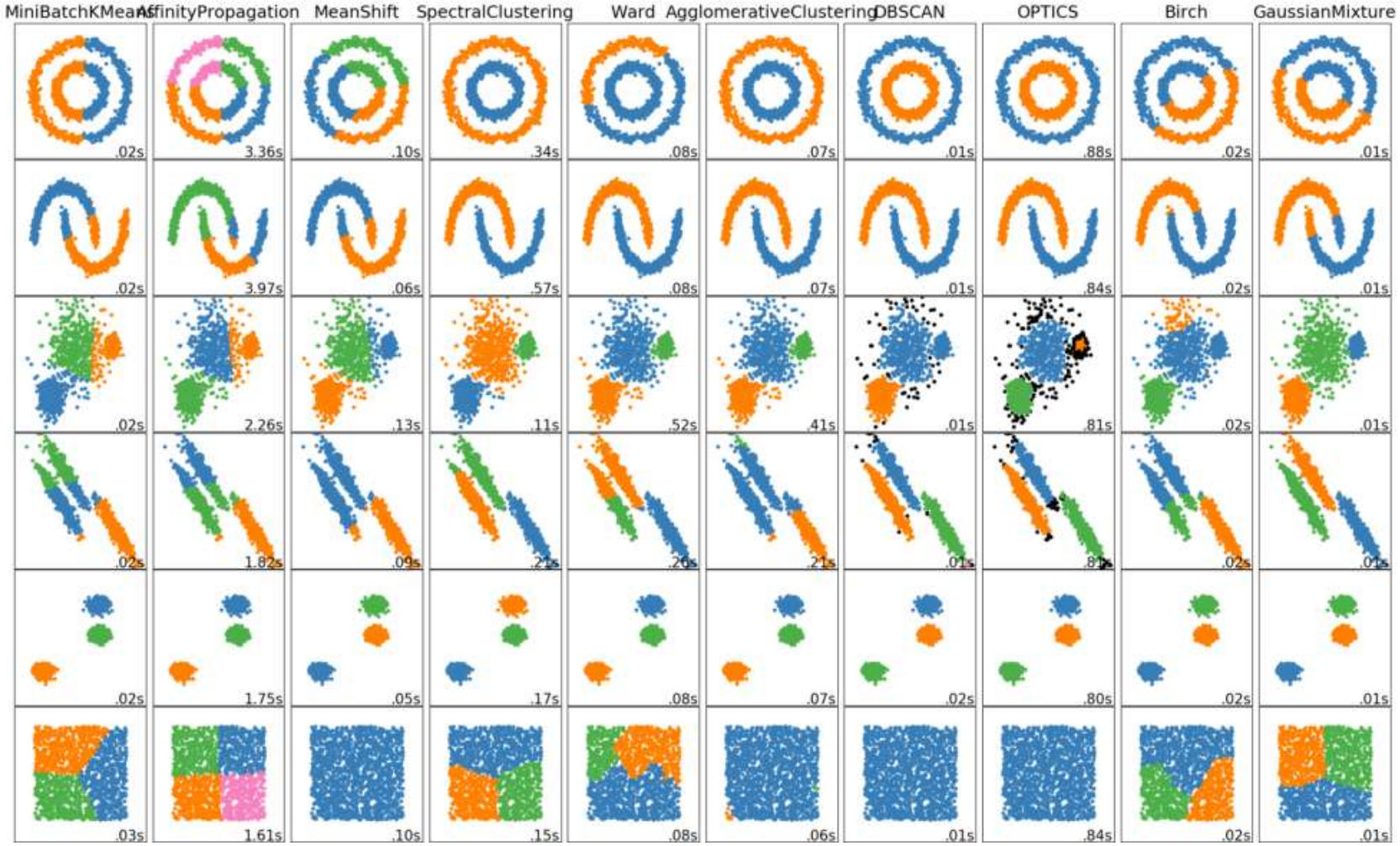


CLUSTERING

- Otros algoritmos estudian cada uno de los puntos y van haciendo crecer los clusters
  - DBSCAN
  - Spectral Clustering (en cierto modo)



CLUSTERING


MiniBatchKMeans
AffinityPropagation
MeanShift
SpectralClustering
Ward
AgglomerativeClustering
DBSCAN
OPTICS
Birch
GaussianMixture
.02s 3.36s .10s .34s .08s .07s .01s .88s .02s .01s
.02s 3.97s .06s .57s .08s .07s .01s .84s .02s .01s
.02s 2.26s .13s .11s .52s .41s .01s .81s .02s .01s
.02s 1.82s .09s .21s .26s .21s .01s .81s .02s .01s
.02s 1.75s .05s .17s .08s .07s .02s .80s .02s .01s
.03s 1.61s .10s .15s .08s .06s .01s .84s .02s .01s

CLUSTERING

Dada una colección de registros/muestras (conjunto de entrenamiento)… Los diferentes algoritmos tratan de responder las siguientes preguntas:

- ¿Qué variables utilizo para hacer el clustering? → Selección de variables
- ¿Cuántos clusters utilizar?
- ¿Cómo medir su calidad?
- ¿Cómo detectar los outliers?
- ¿Es consistente este clustering? (Muchos algoritmos no son deterministas ☹)

Clustering

- **Partition**: Instances are assigned to groups so that those that belong to the same group have higher **similiarity** (**K-means**, SOM, PAM, K-medioids, CLARA, CLARANS, etc.). Some of them can also apply fuzzy theory (instance no longer belong exclusively to one class).

- **Hierarchical**: Instances are joined in a **binary tree according to their similarity**.

- **Distribution (Model-based methods)**: The data are assumed to be originated from a distribution that is estimated (GMM HMM, LDA, EM).

- **Density-Based**: We group the instances according to the density of the distances between them (DBSCAN, DENCLUE, HDBSCAN,...).

- **Graph Theory**: Instances are grouped together forming a graph and later on, the graph is broken into pieces (spectral clustering)
https://www.datanovia.com/en/blog/types-of-clustering-methods-overview-and-quick-start-r-code/

Clustering Distance Measures

Euclidean distance:

$$d_{euc}(x,y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Manhattan distance:

$$d_{man}(x,y) = \sum_{i=1}^{n} |(x_i - y_i)|$$

Pearson correlation distance:

$$d_{cor}(x,y) = 1 - \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n} (x_i - \bar{x})^2 \sum_{i=1}^{n} (y_i - \bar{y})^2}}$$

$$d_{spear}(x,y) = 1 - \frac{\sum_{i=1}^{n} (x_i' - \bar{x}')(y_i' - \bar{y}')}{\sqrt{\sum_{i=1}^{n} (x_i' - \bar{x}')^2 \sum_{i=1}^{n} (y_i' - \bar{y}')^2}}$$

Where $x_i' = \text{rank}(x_i)$ and $y_i' = \text{rank}(y_i)$

Clustering – K-means

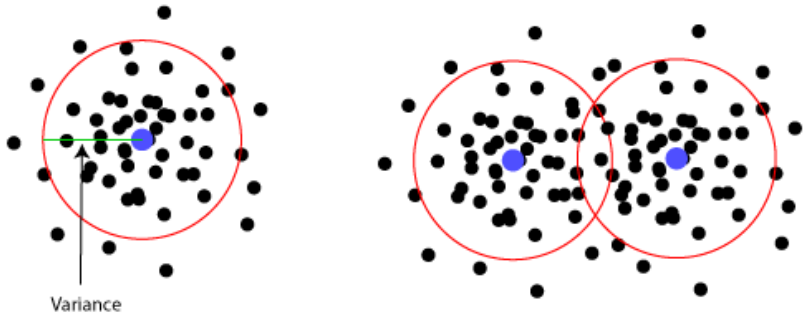
K-means clustering is the simplest and the most commonly used clustering method for splitting a dataset into a set of k groups.

- objects within the same cluster are as similar as possible (intra-class similarity)
- objects from different clusters are as dissimilar as possible (inter-class similarity)
- each cluster is represented by its center (i.e, centroid) which corresponds to the mean of points assigned to the cluster

total within-cluster variation

$$W(C_k) = \sum_{x_i \in C_k} (x_i - \mu_k)^2$$

$$\text{tot. withiness} = \sum_{k=1}^{k} W(C_k) = \sum_{k=1}^{k} \sum_{x_i \in C_k} (x_i - \mu_k)^2$$



Clustering – K-means

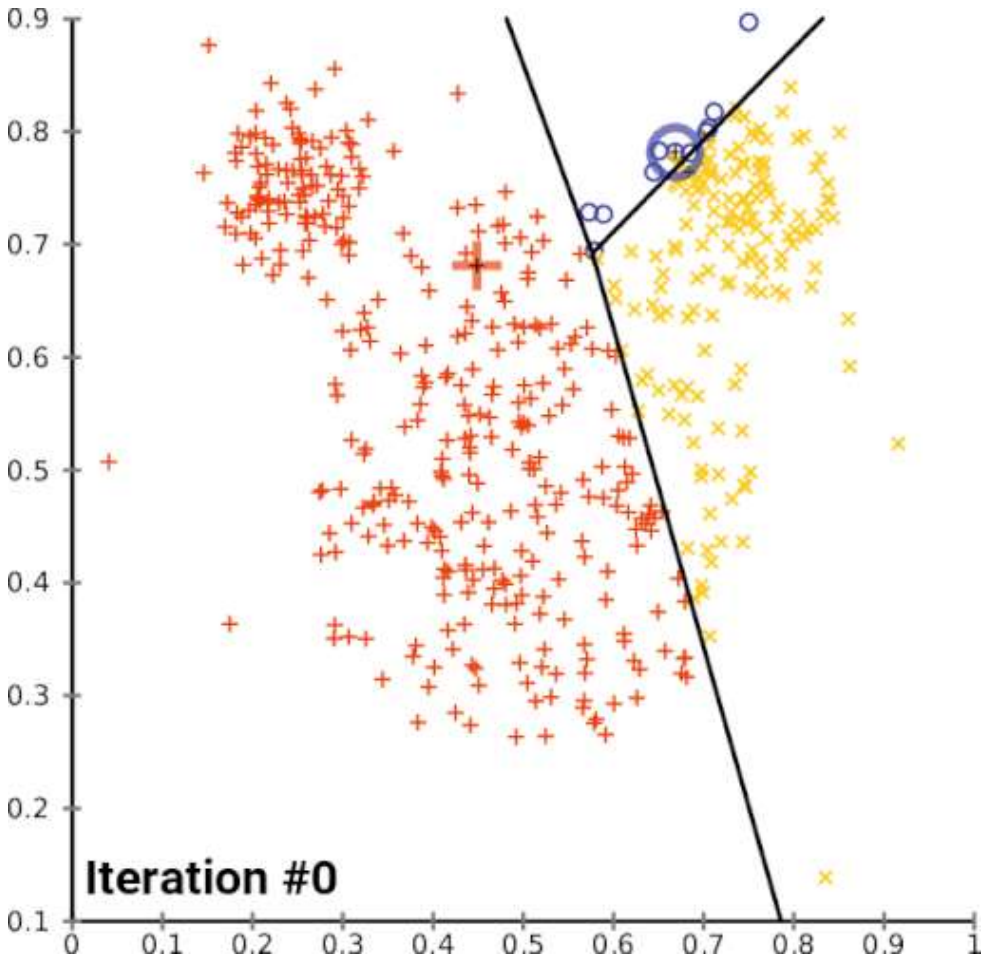
1.Specify the number of clusters (K) to be created (by the analyst).

2.Select randomly k objects as the initial cluster centers or means.

3.Assigns each observation to their closest centroid, based on the Euclidean distance between the object and the centroid

4.For each of the k clusters update the cluster centroid by calculating the new mean values of all the data points in the cluster. The centroid of a Kth cluster is a vector of length *p* containing the means of all variables for the observations in the kth cluster; *p* is the number of variables.

5. Iteratively minimize the total within sum of square. That is, iterate steps 3 and 4 until the cluster assignments stop changing or the maximum number of iterations is reached.



The convergence of the algorithm is not guaranteed. Sometimes is stuck to a local minima.

Clustering – K-means

Advantages	Disadvantages
Simple clustering approach	Requires number of clusters in advance
Efficient	Problems with handling categorical attributes
Cluster size balanced	Does not accodmodate well clusters with different number of instances.
*	Clusters tend to be spherical –> Does not discover clusters with non-convex shapes. *
Does not need a huge number of instances	The distance selection is critical.
Sophisticated initialization methods improve the final results	Convergence not guaranteed for the standard method.

Clustering – best k?

Three most popular methods:

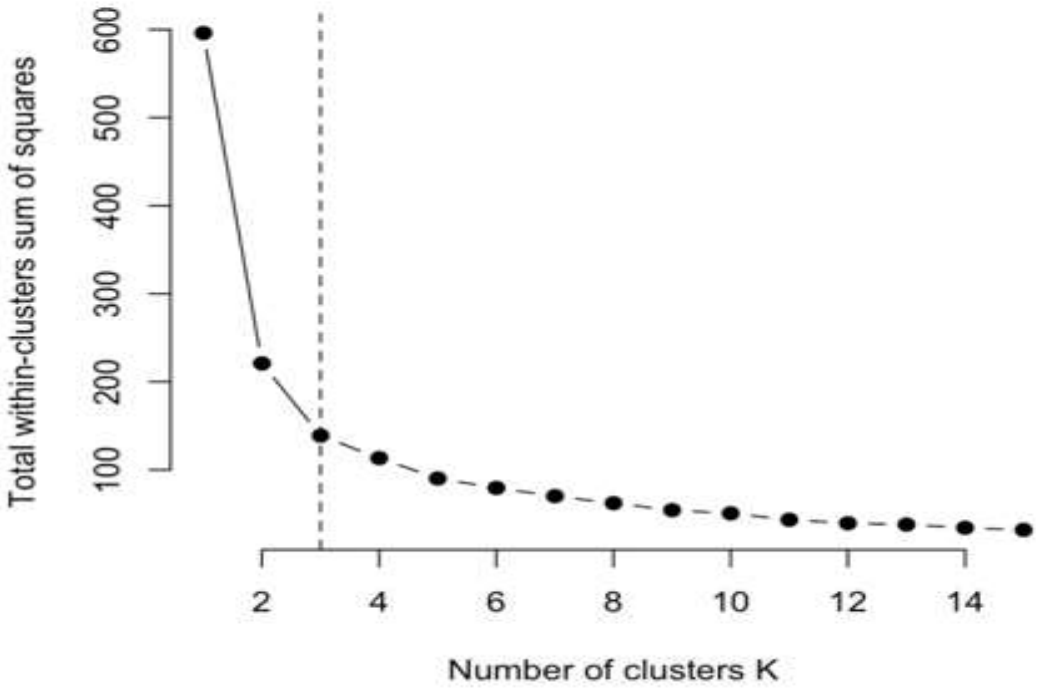
1. Elbow method
2. Silhouette method

1. Compute clustering algorithm (e.g., k-means clustering) for different values of *k*. For instance, by varying *k* from 1 to 10 clusters.

2. For each *k*, calculate the total within-cluster sum of square (wss).

3. Plot the curve of wss according to the number of clusters *k*.

4. The location of a bend (knee) in the plot is generally considered as an indicator of the appropriate number of clusters.



minimize

$$\text{tot. withiness} = \sum_{k=1}^{k} W(C_k)$$

Clustering – best k?

Three most popular methods:

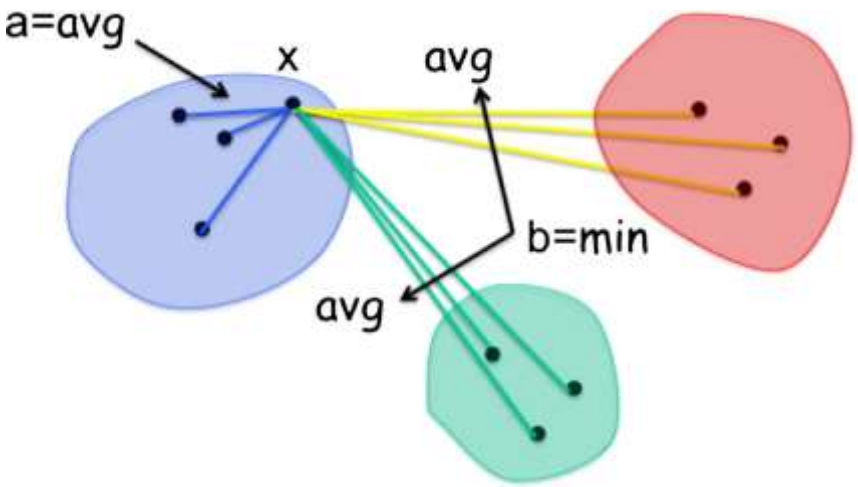
1. Elbow method
2. Silhouette method

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

A **high** average silhouette indicates a **good clustering**.

Cohesion

$$a(i) = \frac{1}{|C_I| - 1} \sum_{j \in C_I, i \neq j} d(i, j)$$



Separation

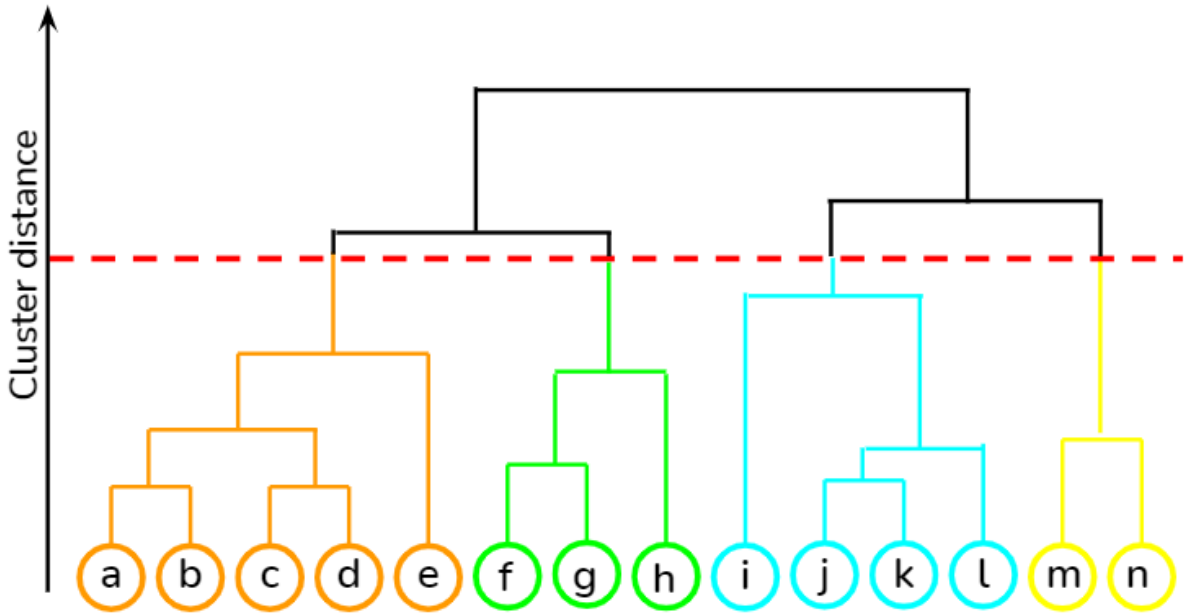
$$b(i) = \min_{J \neq I} \frac{1}{|C_J|} \sum_{j \in C_J} d(i, j)$$

Clustering - Hierarchical

•Clusters at each level of the hierarchy are created by merging clusters at the next lower level. **Once clusters are computed, we get the one which has a lower distance and recompute**.

•Most common strategy is **agglomerative**: start at the bottom and at each level recursively merge a selected pair of clusters into a single cluster. Eventually, we obtain a binary tree (**dendrogram**).

•The height of each node is proportional to the value of the intergroup dissimilarity between its two daughters.



Clustering - Hierarchical

When our clusters have more than one element, the way for computing distances between clusters is defining a strategy to handle the distances of each element between clusters. We can:

- Define the distance mean
- Define the smallest distance
- Define weights and do the weighted sum of distances

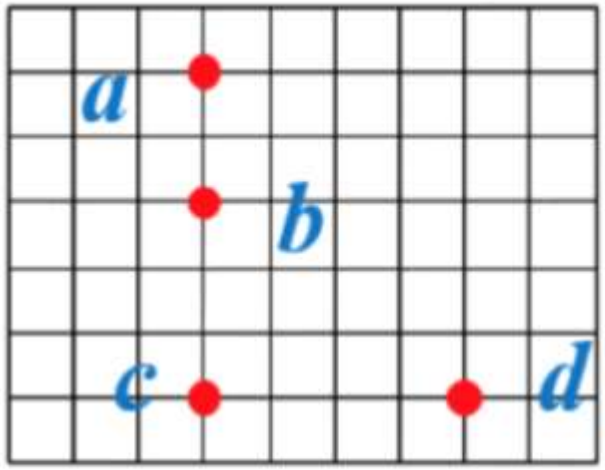
Dissimilarity between two clusters G and H:

- **Simple linkage**: minimal distance between elements of G and H
- **Complete linkage**: maximal distance between elements of G and H
- **Average linkage**: mean distance between elements of G and H
- **UPGMA** (Unweighted Pair Group Method with Arithmetic Mean): distance between the centroids of the cluster.

- **Simple linkage**: Chaining.
- **Complete linkage**: Spherical clusters
- **Average linkage and UPGMA**: Similar results and intermediate compared with previous ones

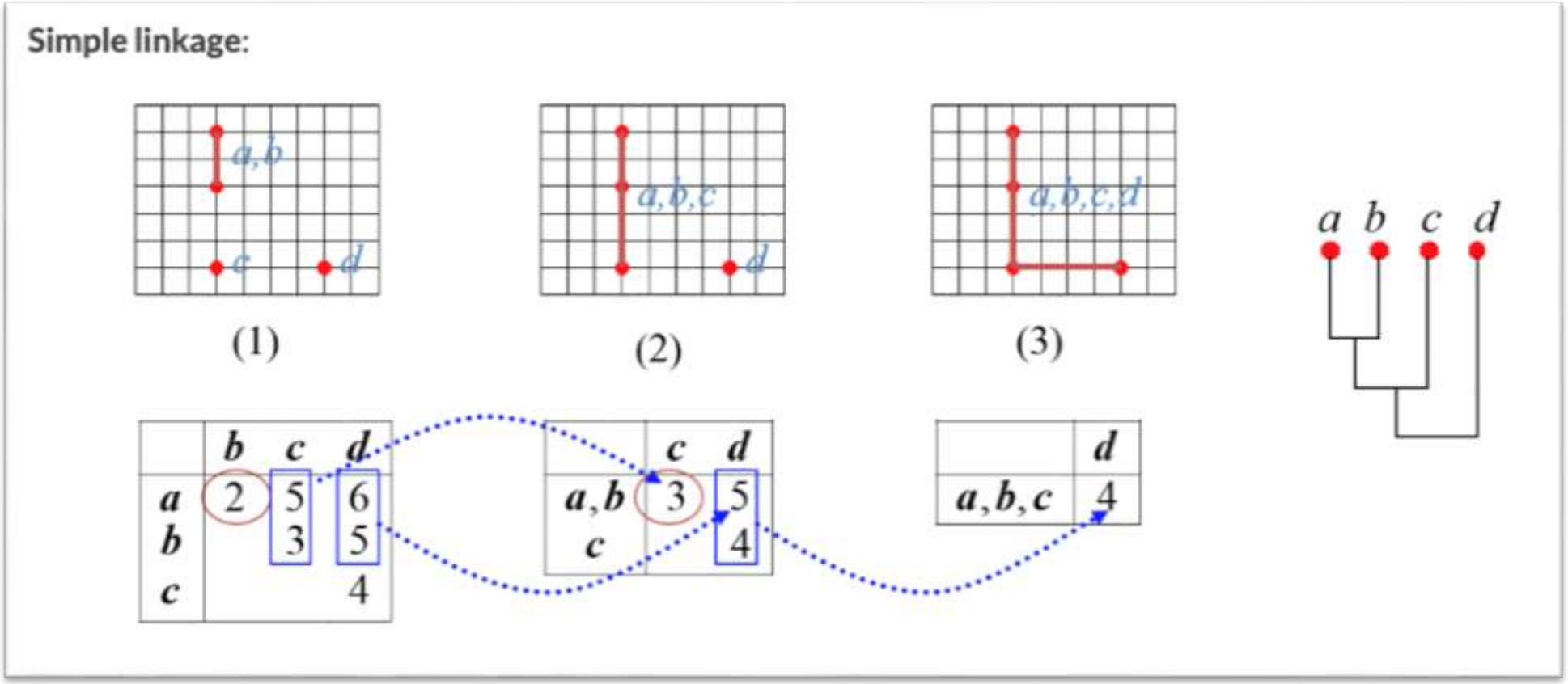
Clustering - Hierarchical

Euclidean Distance



	b	c	d
a	2	5	6
b		3	5
c			4

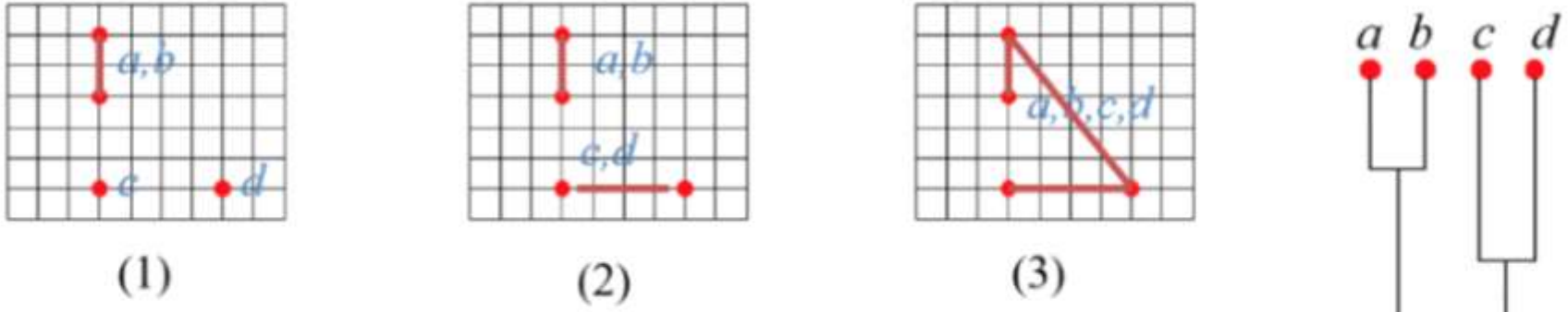
Clustering - Hierarchical



Clustering - Hierarchical

Complete linkage:

Euclidean Distance



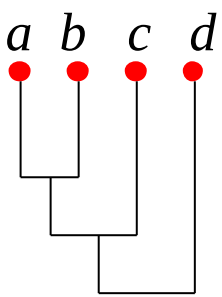
	b	c	d
a	2	5	6
b		3	5
c			4

	c	d
a,b	5	6
c		4

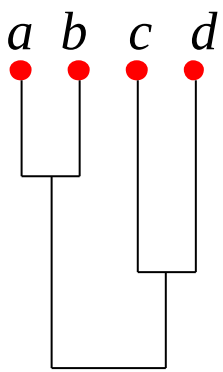
	c,d
a,b	6

Clustering - Hierarchical

Single linkage

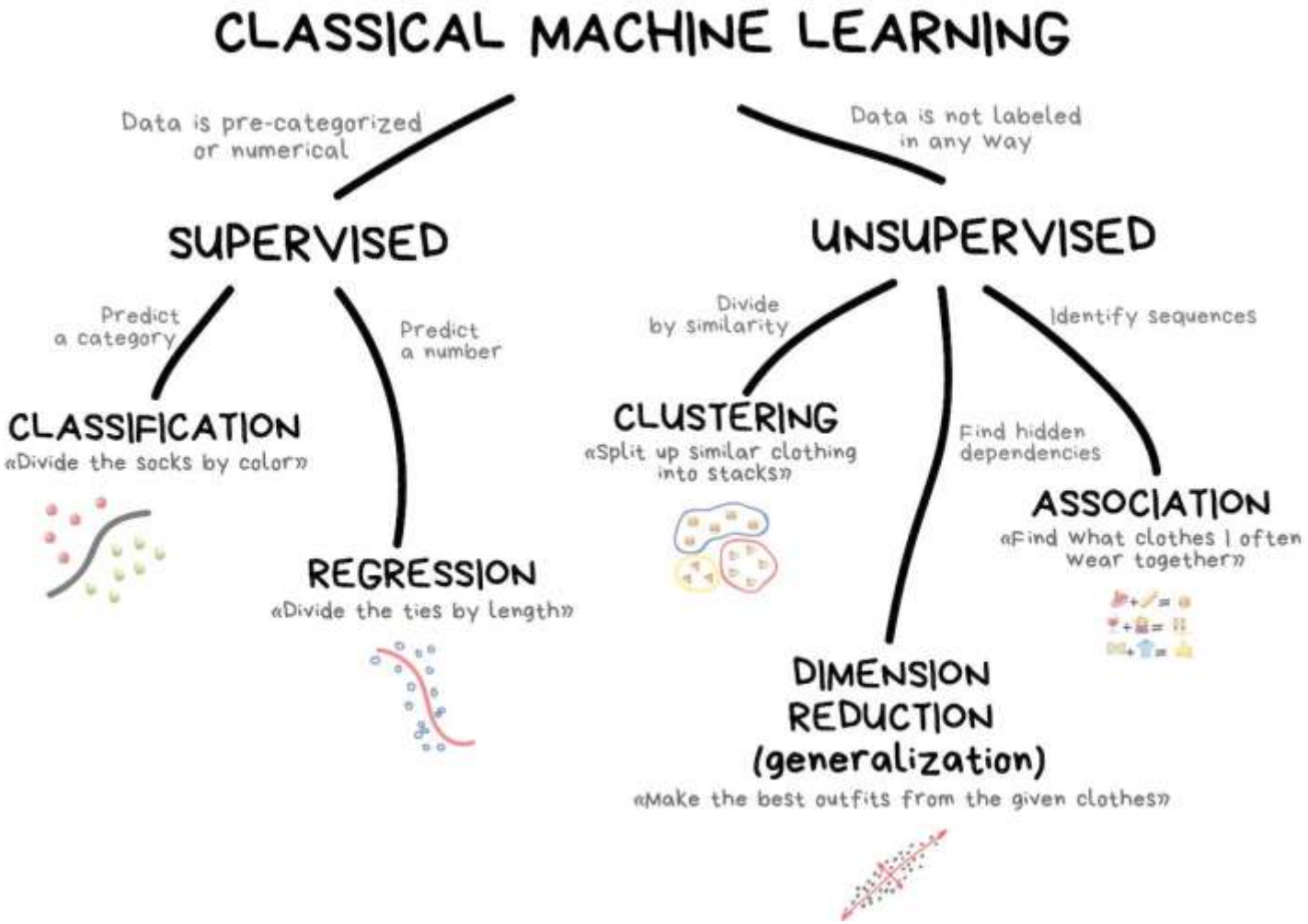


Complete linkage

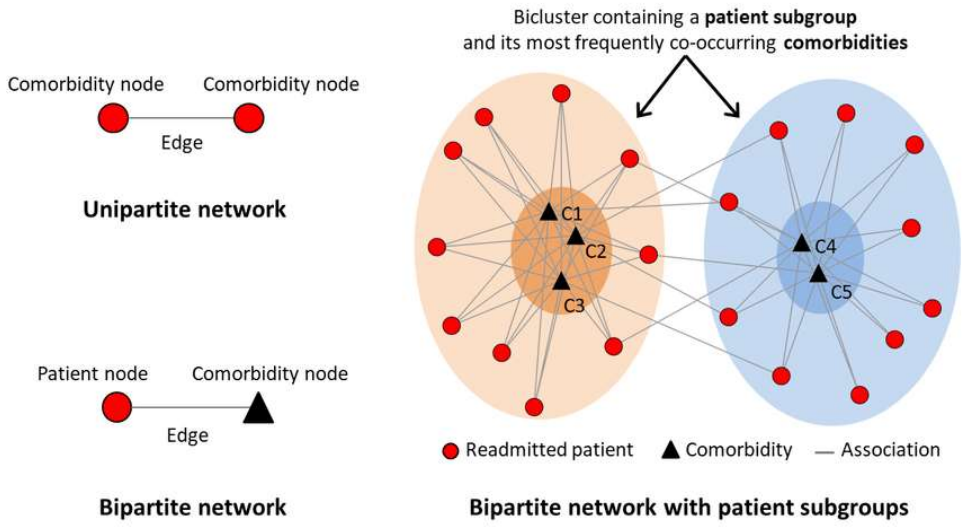


- **Simple linkage**: Chaining.
- **Complete linkage:** Discard more compact groups to cluster
- **Average linkage**: Similar results and intermediate compared with previous ones

Introduction to Machine Learning



Reglas de asociación y Biclustering



Reglas de asociación: “las personas que compran esto, también compran esto otro” o “las personas que tienen estos trastornos también tienen estos otros”

Un algoritmo de “biclustering” encontrar grupos de personas y características que tienen algo en común

- Las reglas de asociación son otra forma de entender el problema: “los que compran esto, también compran esto otro”.
- Puede servir como mecanismo de sugerencia de compra, por ejemplo.

Recommender systems

A recommender system, or a recommendation system (platform or engine), is a subclass of information filtering system that seeks to predict the “rating” or “preference” a user would give to an item.

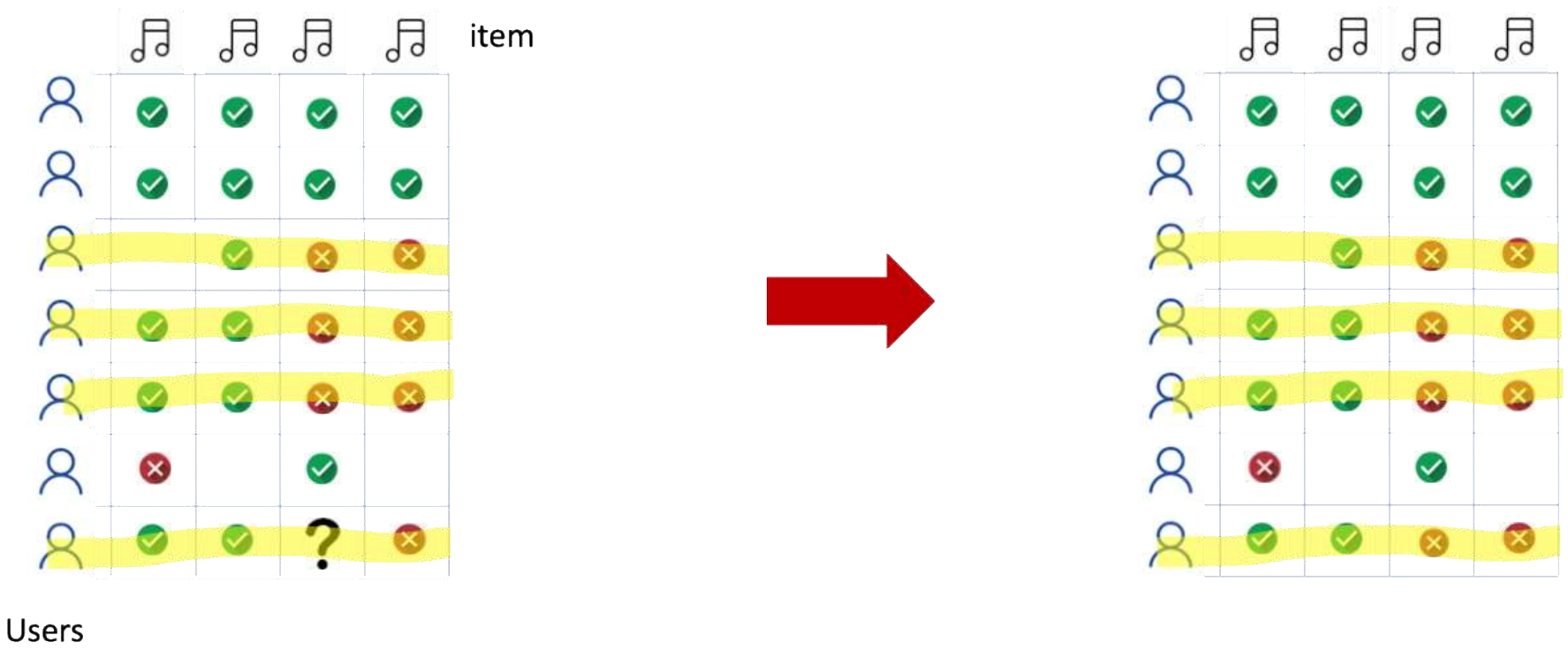
Recommender systems are used in a variety of areas, with commonly recognised examples taking the form of playlist generators for video and music services, product recommenders for online stores, or content recommenders for social media platforms and open web content recommenders (from Wikipedia).

- Collaborative: “Find users like me and tell me what they liked”: Last.fm creates a “station” of recommended songs by observing what bands and individual tracks the user has listened to on a regular basis and comparing those against the listening behavior of other users. Last.fm will play tracks that do not appear in the user’s library, but are often played by other users with similar interests. As this approach leverages the behavior of users, it is an example of a collaborative filtering technique.

- Content-Based: “Find songs similar to the ones I like”: Pandora uses the properties of a song or artist (a subset of the 400 attributes provided by the Music Genome Project) to seed a “station” that plays music with similar properties. User feedback is used to refine the station’s results, deemphasizing certain attributes when a user “dislikes” a particular song and emphasizing other attributes when a user “likes” a song. This is an example of a content-based approach (from Wikipedia).

Recommender systems

- **Collaborative**: "Find users like me and tell me what they liked": Last.fm creates a "station" of recommended songs by observing what bands and individual tracks the user has listened to on a regular basis and comparing those against the listening behavior of other users. Last.fm will play tracks that do not appear in the user's library, but are often played by other users with similar interests. As this approach leverages the behavior of users, it is an example of a collaborative filtering technique.



Recommender systems

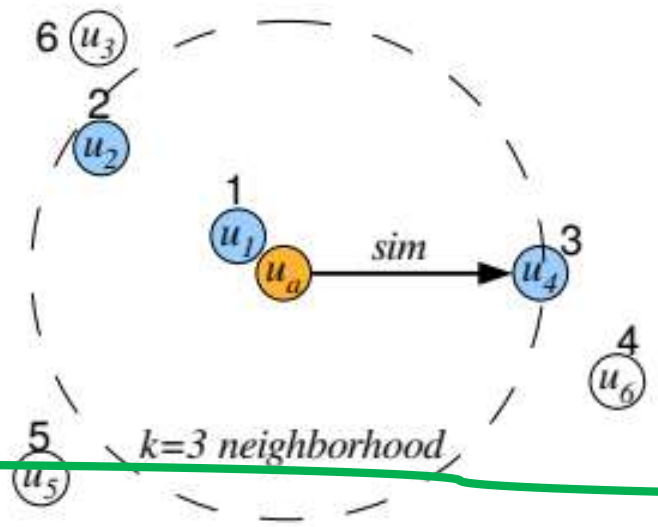
- Collaborative: “Find users like me and tell me what they liked”

User-based Collaborative Filtering (UBCF)

R

	i_1	i_2	i_3	i_4	i_5	i_6	i_7	i_8
u_1	?	4.0	4.0	2.0	1.0	2.0	?	?
u_2	3.0	?	?	?	5.0	1.0	?	?
u_3	3.0	?	?	3.0	2.0	2.0	?	3.0
u_4	4.0	?	?	2.0	1.0	1.0	2.0	4.0
u_5	1.0	1.0	?	?	?	?	?	1.0
u_6	?	1.0	?	?	1.0	1.0	?	1.0
u_a	?	?	4.0	3.0	?	1.0	?	5.0
$\hat{r}_a$	3.5	4.0			2.3		2.0	

s_a	u_a
u_1	0.3
u_2	1.0
u_3	0.2
u_4	0.3
u_5	0.1
u_6	0.1



$$\hat{r}_{al} = \frac{1}{|\mathcal{N}(a)|} \sum_{i \in \mathcal{N}(a)} r_{il}$$

Assumption: is that users with similar preferences will rate items similarly.

missing ratings for a user can be predicted by first finding a neighborhood of similar users and then aggregate the ratings of these users to form a prediction.

Recommender systems

- Collaborative: “Find users like me and tell me what they liked”

User-based Collaborative Filtering (UBCF)

When building a model from a user’s behavior, a distinction is often made between explicit and implicit forms of data collection.

- Explicit: ask user to rate items
- Implicit: observing items viewed by the user. Analysis item/user viewing times, etc..

Problems:

•*Cold start*: For a new user or item, there isn’t enough data to make accurate recommendations.

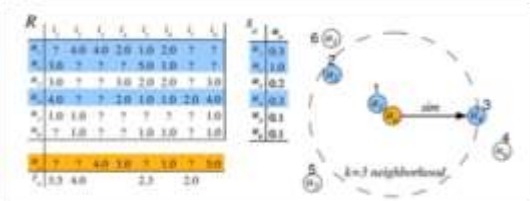
•*Scalability*: In many of the environments in which these systems make recommendations, there are millions of users and products. Thus, a large amount of computation power is often necessary to calculate recommendations.

•*Sparsity*: The number of items sold on major e-commerce sites is extremely large. The most active users will only have rated a small subset of the overall database. Thus, even the most popular items have very few ratings.

Recommender systems

- Collaborative: “Find users like me and tell me what they liked”

Item-based Collaborative Filtering (IBCF)



In the UBCF we used the rows information

S	i_1	i_2	i_3	i_4	i_5	i_6	i_7	i_8	$\hat{r}_a$
i_1	-	0.1	0	**0.3**	**0.2**	**0.4**	0	0.1	-
i_2	0.1	-	**0.8**	**0.9**	0	**0.2**	0.1	0	0.0
i_3	0	**0.8**	-	0	**0.4**	0.1	0.3	**0.5**	4.6
i_4	**0.3**	**0.9**	0	-	0	0.1	0	**0.2**	3.2
i_5	**0.2**	0	**0.4**	0	-	0.1	**0.2**	0.1	-
i_6	**0.4**	**0.2**	0.1	**0.3**	0.1	-	0	0.1	2.0
i_7	0	**0.1**	**0.3**	0	**0.2**	0	-	0	4.0
i_8	**0.1**	0	**0.5**	**0.2**	0.1	0.1	0	-	-
u_a	2	?	?	?	4	?	?	5	

2*0 + 4*0.4 + 5*0.5 = 4.1

$$\hat{r}_{al} = \frac{1}{\sum_{i \in \mathcal{S}(l)} s_{li}} \sum_{i \in \mathcal{S}(l)} s_{li} r_{ai}$$

0 + 0.4 + 0.5 = 0.9

Recommender systems

- Collaborative: “Find users like me and tell me what they liked”

Item-based Collaborative Filtering (IBCF)

Problems:

•*Cold start*: A new item cannot be recommended.

•*Scalability*: Beter than UBCF (less items than users)

•*Sparsity*: The number of items sold on major e-commerce sites is extremely large. The most active users will only have rated a small subset of the overall database. Thus, even the most popular items have very few ratings.

Recommender systems

User-based Collaborative Filtering (UBCF)

R	i_1	i_2	i_3	i_4	i_5	i_6	i_7	i_8
u_1	?	4.0	4.0	2.0	1.0	2.0	?	?
u_2	3.0	?	?	?	5.0	1.0	?	?
u_3	3.0	?	?	3.0	2.0	2.0	?	3.0
u_4	4.0	?	?	2.0	1.0	1.0	2.0	4.0
u_5	1.0	1.0	?	?	?	?	?	1.0
u_6	?	1.0	?	?	1.0	1.0	?	1.0
u_a	?	?	4.0	3.0	?	1.0	?	5.0
$\hat{r}_a$	3.5	4.0			2.3		2.0	

S_a	u_a
u_1	0.3
u_2	1.0
u_3	0.2
u_4	0.3
u_5	0.1
u_6	0.1

Item-based Collaborative Filtering (IBCF)

S	i_1	i_2	i_3	i_4	i_5	i_6	i_7	i_8	$\hat{r}_a$
i_1	-	0.1	0	**0.3**	**0.2**	**0.4**	0	0.1	-
i_2	0.1	-	**0.8**	**0.9**	0	**0.2**	0.1	0	0.0
i_3	0	**0.8**	-	0	**0.4**	0.1	0.3	**0.5**	4.6
i_4	**0.3**	**0.9**	0	-	0	0.1	0	**0.2**	3.2
i_5	**0.2**	0	**0.4**	0	-	0.1	**0.2**	0.1	-
i_6	**0.4**	**0.2**	0.1	**0.3**	0.1	-	0	0.1	2.0
i_7	0	**0.1**	**0.3**	0	**0.2**	0	-	0	4.0
i_8	**0.1**	0	**0.5**	**0.2**	0.1	0.1	0	-	-
u_a	2	?	?	?	4	?	?	5	

Rating bias where some users tend to always use higher ratings while others tend to use lower ratings.

$$h(r_{jl}) = r_{jl} - \bar{\mathbf{r}}_j$$

$$h^{-1}(h(r_{jl})) = h(r_{jl}) + \bar{\mathbf{r}}_j = r_{jl}$$

Recommender systems

- **Content-Based**: "Find songs similar to the ones I like": Pandora uses the properties of a song or artist (a subset of the 400 attributes provided by the Music Genome Project) to seed a "station" that plays music with similar properties. User feedback is used to refine the station's results, deemphasizing certain attributes when a user "dislikes" a particular song and emphasizing other attributes when a user "likes" a song. This is an example of a content-based approach (from Wikipedia).

Users and items have their known metadata:

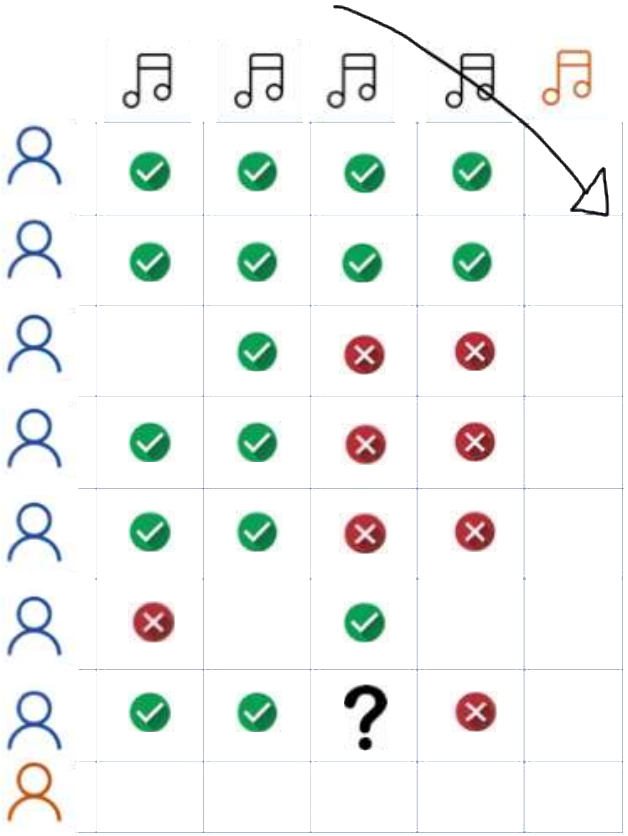
- Users: age, sex, country, etc…
- Item (example of movies): time, year, comedy, drama, etc…

The advantage of this approach is that items metadata are known in advance, so we can also apply it to Cold-Start scenarios where a new item or user is added to the platform and we don't have user-item interactions to train our model.

The disadvantages are that we don't use the full set of known user-item interactions (each user is treated independently), and that we need to know metadata information for each item and user.

content-based filtering only recommends items based on a user's previously evidenced interests, its recommendations are often similar to items a user liked in the past. In other words, CBRSs lack a methodology for exploring the new and unpredicted.

Use the item features



Recommender systems

User-based Collaborative Filtering (UBCF)

R	i_1	i_2	i_3	i_4	i_5	i_6	i_7	i_8
u_1	?	4.0	4.0	2.0	1.0	2.0	?	?
u_2	3.0	?	?	?	5.0	1.0	?	?
u_3	3.0	?	?	3.0	2.0	2.0	?	3.0
u_4	4.0	?	?	2.0	1.0	1.0	2.0	4.0
u_5	1.0	1.0	?	?	?	?	?	1.0
u_6	?	1.0	?	?	1.0	1.0	?	1.0
u_a	?	?	4.0	3.0	?	1.0	?	5.0
$\hat{r}_a$	3.5	4.0			2.3		2.0	

S_a	u_a
u_1	0.3
u_2	1.0
u_3	0.2
u_4	0.3
u_5	0.1
u_6	0.1

Item-based Collaborative Filtering (IBCF)

S	i_1	i_2	i_3	i_4	i_5	i_6	i_7	i_8	$\hat{r}_a$
i_1	-	0.1	0	**0.3**	**0.2**	**0.4**	0	0.1	-
i_2	0.1	-	**0.8**	**0.9**	0	**0.2**	0.1	0	0.0
i_3	0	**0.8**	-	0	**0.4**	0.1	0.3	**0.5**	4.6
i_4	**0.3**	**0.9**	0	-	0	0.1	0	**0.2**	3.2
i_5	**0.2**	0	**0.4**	0	-	0.1	**0.2**	0.1	-
i_6	**0.4**	**0.2**	0.1	**0.3**	0.1	-	0	0.1	2.0
i_7	0	**0.1**	**0.3**	0	**0.2**	0	-	0	4.0
i_8	**0.1**	0	**0.5**	**0.2**	0.1	0.1	0	-	-
u_a	2	?	?	?	4	?	?	5	

Only use the rows or the columns information: can we use both???

MATRIX FACTORIZATION

Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

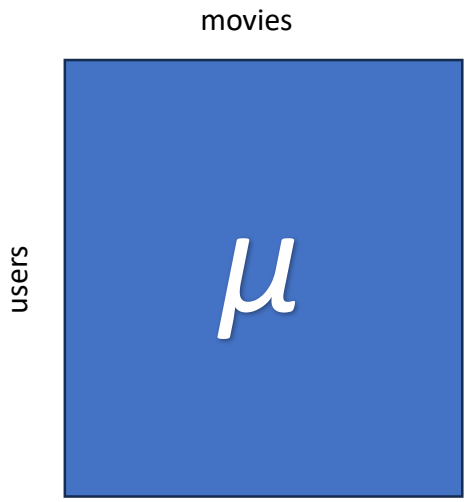
Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

$$\hat{Y}_{u,i} = \mu$$

μ is the mean of observed data and $\epsilon_{i,u}$ is the error distribution

The simplest model predicts all users will give the same rating to all movies



Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

$$\hat{Y}_{u,i} = \mu$$

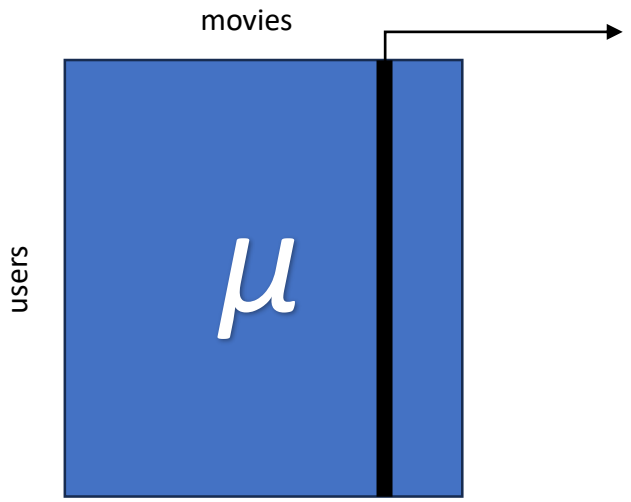
μ is the mean of observed data and $\epsilon_{i,u}$ is the error distribution

The simplest model predicts all users will give the same rating to all movies

$$\hat{y} = \mu + b_i$$

b_i is the movie effect (bias) for movie i.

$$\hat{b_i} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{\mu})$$



Part of the movie variability can be explained by the fact that different movies have different rating distribution.

Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

$$\hat{Y}_{u,i} = \mu$$

μ is the mean of observed data and $\epsilon_{i,u}$ is the error distribution

The simplest model predicts all users will give the same rating to all movies

$$\hat{Y}_{u,i} = \mu + b_i + b_u$$

b_u is the user effect (bias) for user u.

$$\hat{b_u} = \frac{1}{N}\sum_{i=1}^{N}(y_{u,i} - \hat{b_i} - \hat{\mu})$$

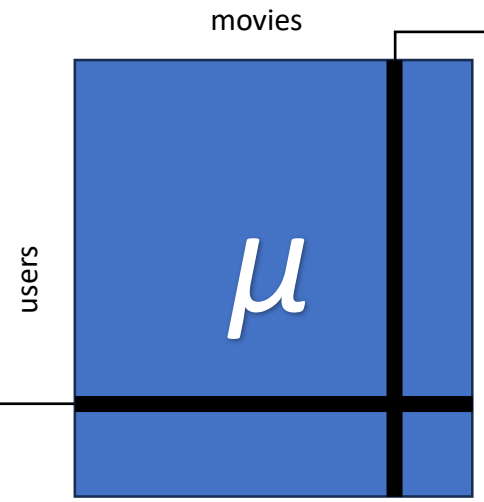
Different users have different rating pattern or distribution. For example, some users like most movies and consistently rate 4 or 5, while other users dislike most movies rating 1 or 2.

$$\hat{y} = \mu + b_i$$

b_i is the movie effect (bias) for movie i.

$$\hat{b_i} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{\mu})$$

Part of the movie variability can be explained by the fact that different movies have different rating distribution.



Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

Notice that the user effect is used to model the residuals, i.e., we computed the mean, find out the residuals, they were modelled according to the movies, then we got the residuals, and they were modelled using the users.
Another possibility, probably more sensible, is to model the whole system as a linear model with two effects: the users and the movie.

$$\hat{Y}_{u,i} = \mu + b_i + b_u$$

b_u is the user effect (bias) for user u.

$$\hat{b_u} = \frac{1}{N}\sum_{i=1}^{N}(y_{u,i} - \hat{b_i} - \hat{\mu})$$

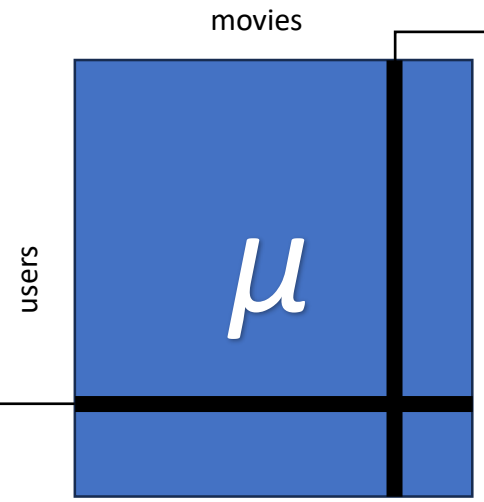
Different users have different rating pattern or distribution. For example, some users like most movies and consistently rate 4 or 5, while other users dislike most movies rating 1 or 2.

$$\hat{y} = \mu + b_i$$

b_i is the movie effect (bias) for movie i.

$$\hat{b_i} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{\mu})$$

Part of the movie variability can be explained by the fact that different movies have different rating distribution.



Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

The linear model doesn't consider that many movies have very few number of ratings, and some users rate very few movies. This means that the sample size is very small for these movies and these users. Statistically, this leads to large estimated error.
The estimated value can be improved adding a factor that penalizes small sample sizes and have have little or no impact otherwise.

$$\hat{Y}_{u,i} = \mu + b_i + b_u$$

b_u is the user effect (bias) for user u.

$$\hat{b_u} = \frac{1}{N}\sum_{i=1}^{N}(y_{u,i} - \hat{b_i} - \hat{\mu})$$

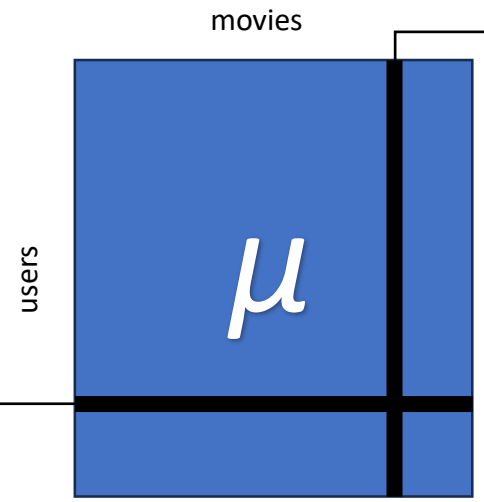
Different users have different rating pattern or distribution. For example, some users like most movies and consistently rate 4 or 5, while other users dislike most movies rating 1 or 2.

$$\hat{y} = \mu + b_i$$

b_i is the movie effect (bias) for movie i.

$$\hat{b_i} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{\mu})$$

Part of the movie variability can be explained by the fact that different movies have different rating distribution.



Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

The linear model doesn't consider that many movies have very few number of ratings, and some users rate very few movies. This means that the sample size is very small for these movies and these users. Statistically, this leads to large estimated error.
The estimated value can be improved adding a factor that penalizes small sample sizes and have have little or no impact otherwise.

$$\hat{Y}_{u,i} = \mu + b_i + b_u$$

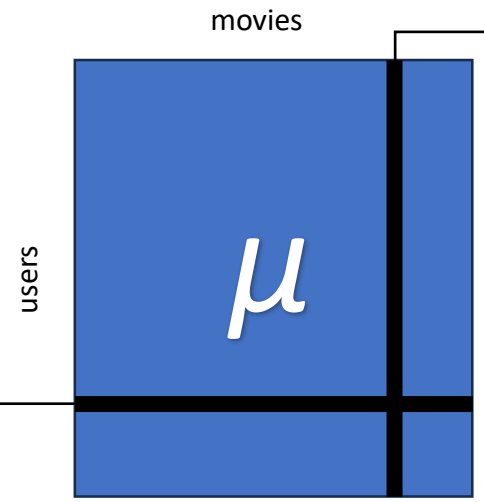
b_u is the user effect (bias) for user u.

$$\hat{b_u} = \frac{1}{n_u + \lambda} \sum_{i=1}^{n_u} (y_{u,i} - \hat{b_i} - \hat{\mu})$$

$$\hat{y} = \mu + b_i$$

b_i is the movie effect (bias) for movie i.

$$\hat{b_i} = \frac{1}{n_i + \lambda} \sum_{u=1}^{n_i} (y_{u,i} - \hat{\mu})$$



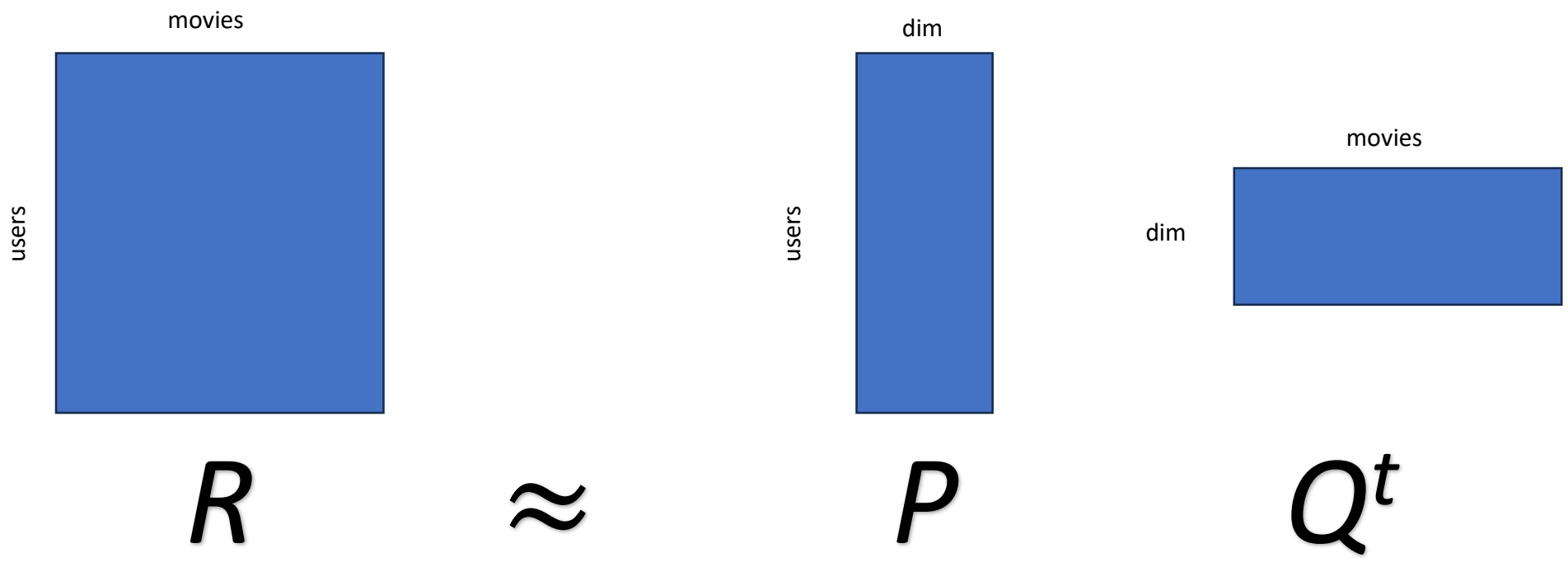
For values of N smaller than or similar to λ, $\hat{b_i}$ and $\hat{b_u}$ is smaller than the original values, whereas for values of N much larger than λ, $\hat{b_i}$ and $\hat{b_u}$ change very little.

Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

Matrix factorization is widely used machine learning tool for predicting ratings in recommendation systems. This method became widely known during the Netflix Prize challenge.

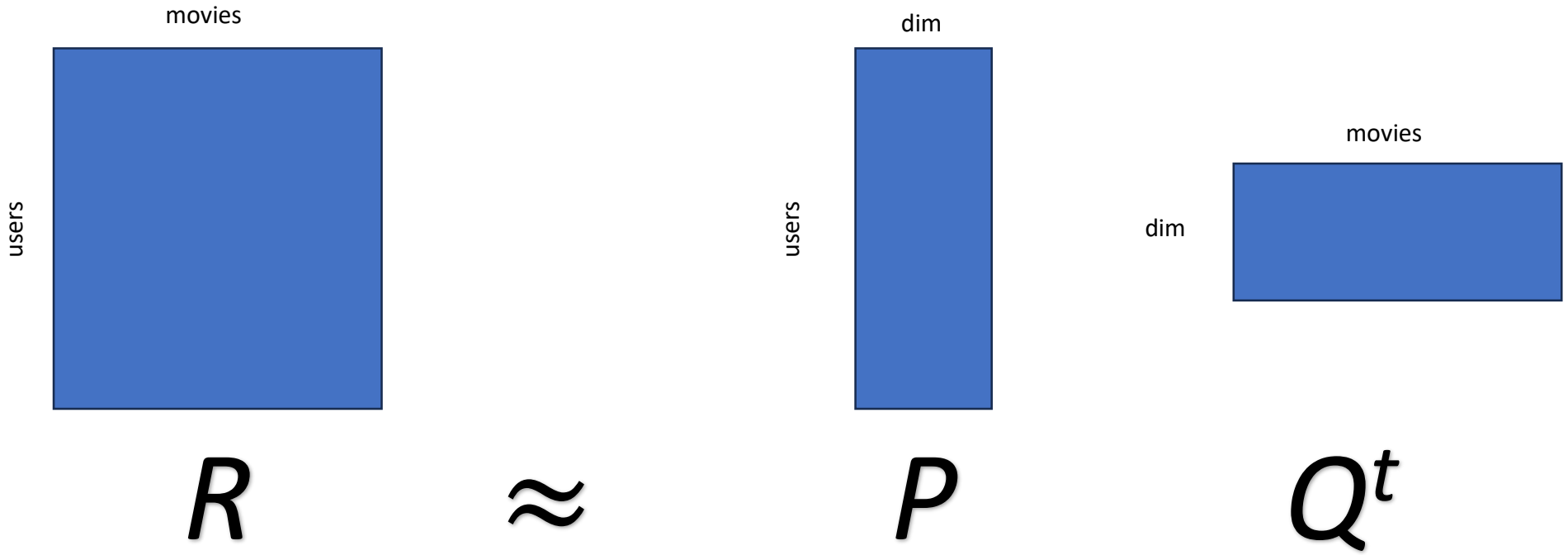
The concept is to approximate a large rating matrix $R_{m \times n}$ into the product of two lower dimension matrices $P_{m\text{xk}}$ and $Q_{n\text{xk}}$



Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

How we obtain P and Q?



Recommender systems

Linear Model.
Regularization.
MATRIX FACTORIZATION.

Limitations:

- memory: the required amount of memory far exceeded the available in a commodity laptop.
- Only two predictors are used, the movie and user information, not considering other features. Modern recommendation system models use many predictors, such as genres, bookmarks, playlists, etc.
- The model works only for existing users, movies and rating values, so the algorithm must run every time a new user or movie is included, or when the rating changes.
- There is no initial recommendation for a new user or for users that usually don’t rate movies. Algorithms that uses several features as predictors can overcome this issue.

Data Science

Introduction to Machine Learning

Dr. Juan A. Ferrer-Bonsoms jafhernandez@unav.es

Tecnun Universidad de Navarra | ESCUELA DE INGENIERÍA INGENIARITZA ESKOLA SCHOOL OF ENGINEERING